UNIT 3 MINI PROJECT

Enterprise Smart Library Management Platform

DevOps Foundation, Version Control, Continuous Integration & Container Orchestration

Name: Om Pandurang Jadhav

PRN No.: 2125UDSM1051

Subject: DevOps

Enterprise Smart Library Management Platform

DevOps Foundation, Version Control, Continuous Integration & Container Orchestration
Document 1: Comprehensive Theoretical and Architectural Reference Manual

———————————————————————————————————————

Document Attribute	Specification Details
Course & Unit	UNIT 3: DevOps, CI/CD, Containerization & Application Deployment
Target Platform	Enterprise Smart Library Management Platform (Python / Flask / Pytest / Jenkins / Docker / Minikube)
Student Parallel System	Enterprise Student Record & Academic Management Platform
Document Purpose	Comprehensive Academic & Professional DevOps Theoretical Framework, Architecture & Engineering Principles
Version & Status	v1.0.0 Production Release — Comprehensive Academic Manual

1. Executive Summary & Engineering Context

In enterprise software engineering, bridging the traditional chasm between application development and operational stability is paramount. Prior academic phases established requirements gathering, backlog grooming, and Agile sprint planning. Unit 3 marks the crucial transition from conceptual design to production-grade engineering practices.

DevOps is fundamentally not merely an assortment of automation utilities; it represents an institutional engineering culture and methodology designed to integrate developers, quality assurance engineers, and IT operations into an agile, highly automated continuum. Rather than treating software deployment as an afterthought, enterprise engineering treats the build, validation, packaging, and hosting pipeline as code.

This document provides an exhaustive theoretical foundation for the Enterprise Smart Library Management Platform. It articulates the business imperatives driving DevOps adoption, contrasts traditional waterfall IT with modern CI/CD, breaks down the complete eight-stage DevOps lifecycle, details distributed version control mechanics in Git/GitHub, explicates Continuous Integration (CI) with Jenkins, and analyzes containerization with Docker and orchestration via Kubernetes (Minikube).

> 📌 **CORE ARCHITECTURAL OBJECTIVE:** DevOps bridges development and operations through Culture, Automation, Measurement, and Sharing (CAMS).
>
> The primary engineering objective is reducing Cycle Time (from commit to customer) while increasing Deployment Frequency, Stability, and Mean Time to Recovery (MTTR).

2. The Business Need for DevOps: Cultural Paradigm Shift

2.1 The Traditional Software Delivery Dilemma

Historically, software engineering organizations operated in rigid organizational silos. The development team was incentivized by business stakeholders to rapidly write new features, while the IT operations team was incentivized to maintain infrastructure stability, uptime, and strict change control.

This misalignment generated what industry pioneers describe as the 'Wall of Confusion'. The traditional sequential delivery model (Waterfall) proceeded through protracted phases:

- **Requirements Silo:** Requirements were locked down months in advance, leaving no room for market agility.
- **Development Silo:** Software engineers worked in isolation, without visibility into production server configurations.
- **Testing Silo:** Large, infrequent batches of code were thrown over the wall to QA for manual verification.
- **Operations Silo:** Sysadmins received release packages with extensive, error-prone manual runbooks.
- **Maintenance Silo:** Unmonitored bugs and performance degradations escalated into midnight outages and stressful blame games.

Key systemic pathologies of traditional IT include:

- **Environment Drift:** The notorious 'Works on My Machine' syndrome, where applications operate on developer workstations but crash on staging or production servers due to subtle environment discrepancies.
- **Integration Hell:** Merging months of unintegrated work resulted in weeks of merge conflicts and regressions.
- **Error-Prone Manual Releases:** Manual SSH scripts, FTP uploads, and manual configuration tweaks yielded high change failure rates.
- **Protracted Downtime & Rollback Risks:** When a production defect emerged, rolling back required recreating complex database states and restoring monolithic servers.

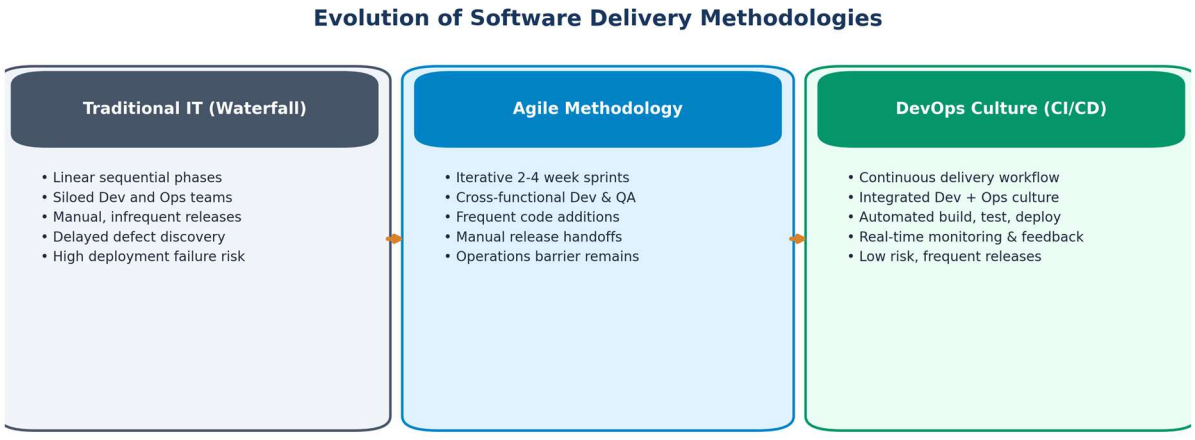


Figure: Evolution of Software Delivery Methodologies from Traditional IT to Agile and DevOps

2.2 Comparative Evolution: Traditional IT vs. Agile vs. DevOps

While Agile principles significantly improved project management and developer responsiveness by breaking monolithic requirements into 2-to-4 week sprints, Agile alone stopped at the boundary of working software on a developer's workstation. It failed to address the downstream friction of deployment, configuration management, and runtime reliability. DevOps completes the transformation.

Engineering Dimension	Traditional IT (Waterfall)	Agile Methodology	DevOps Framework (CI/CD)
Delivery Rhythm	Sequential, monolithic (Months / Quarters)	Iterative sprints (2 to 4 weeks)	Continuous, on-demand (Multiple daily releases)
Organizational Structure	Strictly siloed Dev, QA, and Ops teams	Cross-functional Dev & QA teams	Integrated cross-functional Product & DevOps teams
Deployment Method	Manual checklist, fragile shell scripts	Manual or partially automated at sprint end	Fully automated, immutable CI/CD pipelines
Environment Parity	Frequent drift between Dev, QA, & Prod	Staging environments diverge from Prod	Guaranteed parity via Docker containers & IaC
Quality Verification	Post-development manual regression testing	Sprint-based manual & automated testing	Continuous shift-left automated tests on every commit
Feedback & Telemetry	Periodic customer surveys, post-mortems	Sprint review and retrospective meetings	Real-time automated metrics, logs, and APM alerting
Change Failure & Risk	High blast radius; catastrophic rollbacks	Moderate risk localized to sprint drops	Low blast radius; instant rollback via blue/green or canary

3. The End-to-End DevOps Lifecycle Architecture

The DevOps lifecycle is modeled as an infinite loop rather than a linear sequence, underscoring that modern software engineering is an iterative, continuous feedback mechanism. Every production observation immediately informs future architecture and backlog planning.

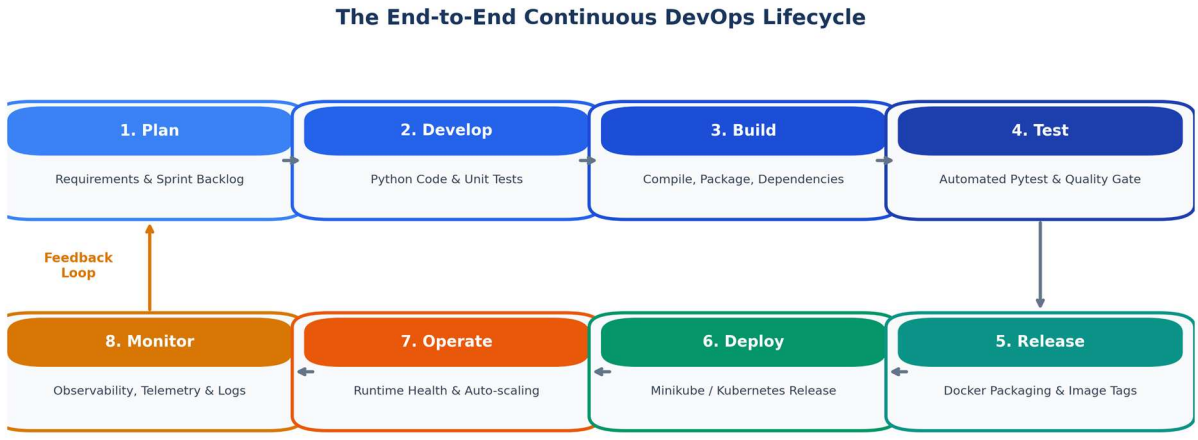


Figure: The Infinite Continuous DevOps Engineering Lifecycle

3.1 Comprehensive Lifecycle Phase Breakdown

- **Phase 1 - Plan:** Engineers and product owners analyze business requirements, user stories, and acceptance criteria. Tasks are broken into small increments and tracked via backlogs (Jira, GitHub Issues, Kanban boards). Architectural standards and non-functional requirements (security, scalability) are defined.
- **Phase 2 - Develop (Code):** Engineers write modular, clean Python application code within integrated development environments (IDEs). Code is version-controlled in short-lived Git feature branches. Pair programming, automated local linting (flake8, black), and pre-commit hooks ensure baseline quality.
- **Phase 3 - Build:** Automated builds compile code, isolate runtime dependencies, and assemble immutable build artifacts. In Python, this involves creating clean virtual environments, resolving requirements.txt, and validating syntax across all packages.
- **Phase 4 - Test:** Continuous Integration servers execute rigorous automated unit tests, integration tests, and static code quality analysis (pytest, coverage, SonarQube). Builds fail immediately if any test fails ('Shift-Left Testing').
- **Phase 5 - Release:** Validated artifacts are bundled into versioned Docker container images, tagged with semantic version numbers (e.g., v1.0.0, commit SHA), and pushed to an enterprise container registry (Docker Hub, Amazon ECR, Harbor).
- **Phase 6 - Deploy:** The containerized application is deployed to target staging or production runtime infrastructure (Kubernetes clusters, Minikube). Deployments utilize rolling updates to ensure zero downtime for end users.
- **Phase 7 - Operate:** Infrastructure and application health are managed actively. Auto-scalers dynamically adjust replica counts based on CPU/memory pressure. Database connection pools, caching layers, and load balancers maintain optimal throughput.
- **Phase 8 - Monitor:** Distributed tracing, metrics collection, and centralized log aggregation (Prometheus, Grafana, ELK Stack, Datadog) monitor runtime performance. Automated synthetic health checks and alert thresholds trigger instant root-cause analysis.

> 📌 **SHIFT-LEFT TESTING PARADIGM:** Shift-Left Principle: The financial and operational cost of defect remediation increases by 10x with each lifecycle stage.
>
> Catching bugs during local development or automated CI testing costs fractions of cents compared to diagnosing a high-severity incident in live production.

4. Enterprise Version Control & Collaborative Git Workflows

4.1 Distributed Version Control Principles

Version Control Systems (VCS) record changes to files over time, enabling software engineers to recall specific versions, audit changes, and collaborate concurrently without destructive overwrites. Git, engineered by Linus Torvalds, is a Distributed Version Control System (DVCS). Unlike centralized systems (such as SVN or Perforce), every developer's local machine hosts a complete clone of the repository history.

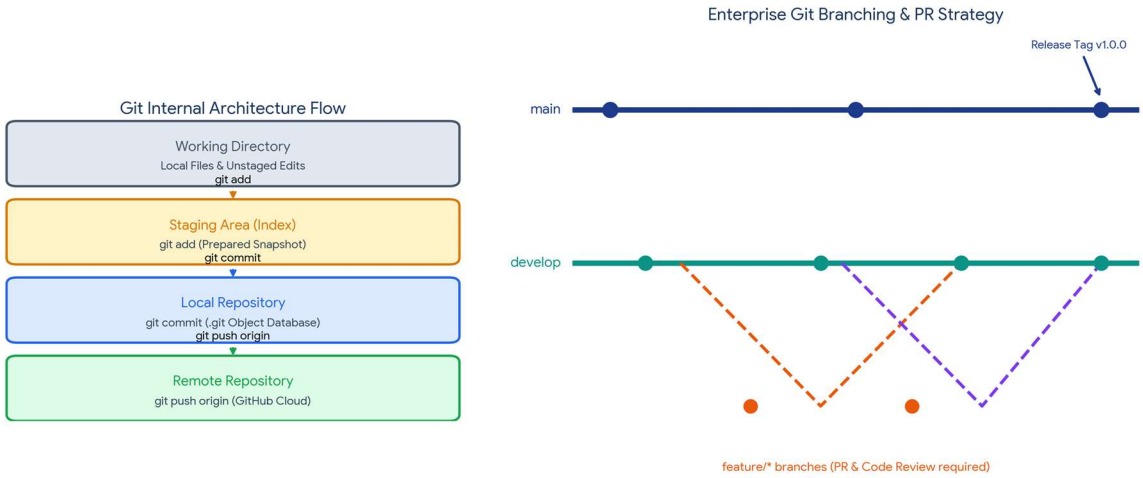


Figure: Git Internal Architectural Flow and Enterprise Feature-Branching Strategy

4.2 Git Three-Tier Architecture

Git manages files across three logical areas on a developer's workstation before synchronizing with remote repositories:

- **1. Working Directory:** The physical file directory containing project code, configs, and assets where active editing occurs.
- **2. Staging Area (Index):** A staging index (the .git/index file) that acts as a draft snapshot where selected modifications are prepared using 'git add' prior to committing.
- **3. Local Repository (.git):** The hidden .git directory containing the object database (blobs, trees, commits, tags). Running 'git commit' permanently records an immutable SHA-1 hashed snapshot.
- **4. Remote Repository:** A centralized hosting platform (GitHub, GitLab, Bitbucket) that acts as the single source of truth for the entire distributed team, updated via 'git push'.

4.3 Comparative Matrix: Git vs. GitHub

Criterion	Git (Core Engine)	GitHub (Collaboration Platform)
Classification	Open-source distributed version control CLI tool	Cloud-based hosting, collaboration, and DevOps platform
Execution Location	Runs entirely locally on the developer's computer	Hosted on cloud infrastructure (web GUI, REST APIs)
Network Dependency	Completely offline; commits, logs, branches need no internet	Requires internet access to push, pull, review, or trigger webhooks
Primary Capabilities	Branching, merging, commit tracking, diff inspection	Pull requests, code review, issue tracking, project boards, actions
Enterprise CI/CD Role	Version records source files and pipeline definitions	Houses source code and triggers Jenkins CI/CD via webhooks

4.4 Enterprise Branching Strategies

A structured branching strategy prevents unstable experimental code from polluting production systems. For enterprise platforms, a controlled Feature Branch Workflow is industry standard:

- **main (Production Branch):** Strictly protected branch reflecting production-ready, thoroughly validated code. Direct pushes are barred; merges require approved Pull Requests.
- **develop (Development Branch):** Integration branch where completed feature branches converge for end-to-end integration and staging validation.
- **feature/* (Feature Branches):** Short-lived branches branched off develop (e.g., feature/user-auth, feature/book-catalog). Developers commit frequently in isolation, then open a Pull Request.
- **hotfix/* (Urgent Fixes):** Emergency branches branched directly from main to patch critical production defects, followed by immediate merge-back into both main and develop.

5. Continuous Integration (CI) and Jenkins Build Automation

5.1 Theoretical Foundation of Continuous Integration

Continuous Integration is the software engineering discipline where team members integrate their work into the main codebase frequently (typically multiple times a day). Each integration is automatically verified by an automated build and test sequence.

Without CI, teams suffer from 'Integration Hell'—a crippling condition where developers work in isolation for weeks, leading to massive merge conflicts, incompatible interface assumptions, and widespread breakage that halts progress.

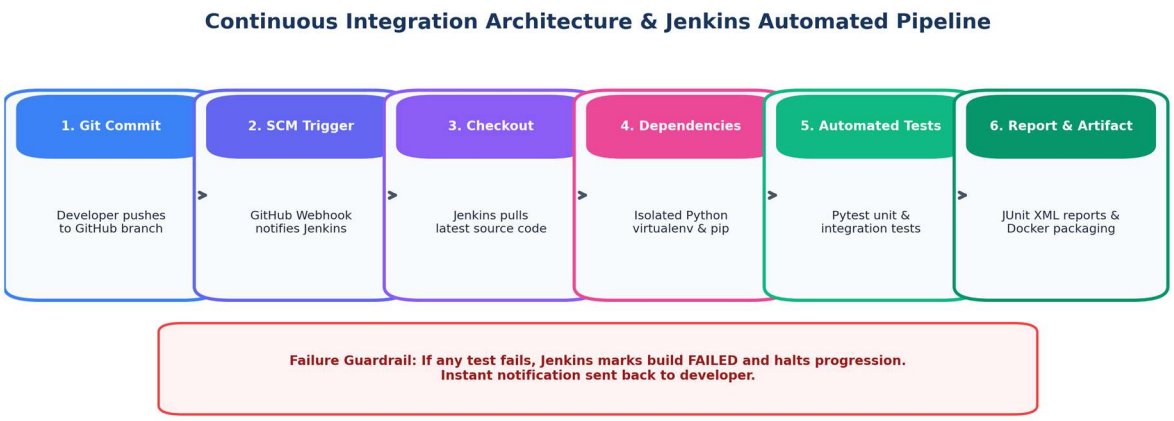


Figure: Continuous Integration Architecture & Jenkins Automated Pipeline Workflow

5.2 Anatomy of the CI Pipeline Stages

- **Stage 1 - Source Checkout:** Jenkins receives an automated webhook trigger from GitHub indicating a new push or pull request, and securely clones the exact commit SHA.
- **Stage 2 - Dependency Isolation:** An isolated Python virtual environment is instantiated. The package manager (pip) installs exact library versions declared in requirements.txt.
- **Stage 3 - Linting & Static Code Analysis:** Static analysis tools (flake8, pylint) verify syntax, check code standards (PEP 8), and catch syntax regressions without running the code.
- **Stage 4 - Automated Test Execution:** The automated test framework (pytest) executes unit and integration tests. Assertions check data models, service business logic, and API endpoints.
- **Stage 5 - Quality Gate & Coverage Analysis:** Test reports are generated in standardized JUnit XML and HTML formats. Code coverage must exceed threshold percentages (e.g., 85%).

- **Stage 6 - Artifact Packaging & Failure Guard:** If all tests and gates succeed, the pipeline packages the deployable artifact (Docker image) or notifies the deployment manager. If any stage fails, the pipeline halts immediately, alerting the committer.

5.3 Jenkins Architecture: Controller and Agent Model

Jenkins is the industry-standard, open-source automation engine. Its enterprise architecture operates on a distributed Controller-Agent pattern:

- **Jenkins Controller (Master):** Hosts the web user interface, manages user authentication, schedules build jobs, monitors agents, and receives webhook notifications from GitHub.
- **Jenkins Build Agents (Workers):** Lightweight worker instances (virtual machines, containers, or bare-metal servers) that execute the actual resource-intensive build, test, and container packaging commands as directed by the controller.

6. Containerization, Docker Architecture & Runtime Packaging

6.1 Resolving Environment Inconsistency: Virtual Machines vs. Containers

Traditional IT deployed applications directly onto bare-metal operating systems or full Virtual Machines (VMs). Virtual machines require a Hypervisor (e.g., VMware, Hyper-V, KVM) that runs an entire Guest Operating System (including kernel, system drivers, and background daemons) for every application, consuming gigabytes of RAM and taking minutes to boot.

Docker revolutionizes software delivery through OS-level virtualization. Containers share the host operating system kernel while isolating execution processes through two fundamental Linux kernel primitives:

- **Linux Namespaces:** Provide strict process isolation by creating distinct virtual views of process IDs (PID), network interfaces (NET), mount points (MNT), and user IDs (USER).
- **Control Groups (cgroups):** Provide resource governance by strictly metering and capping memory, CPU utilization, disk I/O, and network bandwidth allocated to each container.

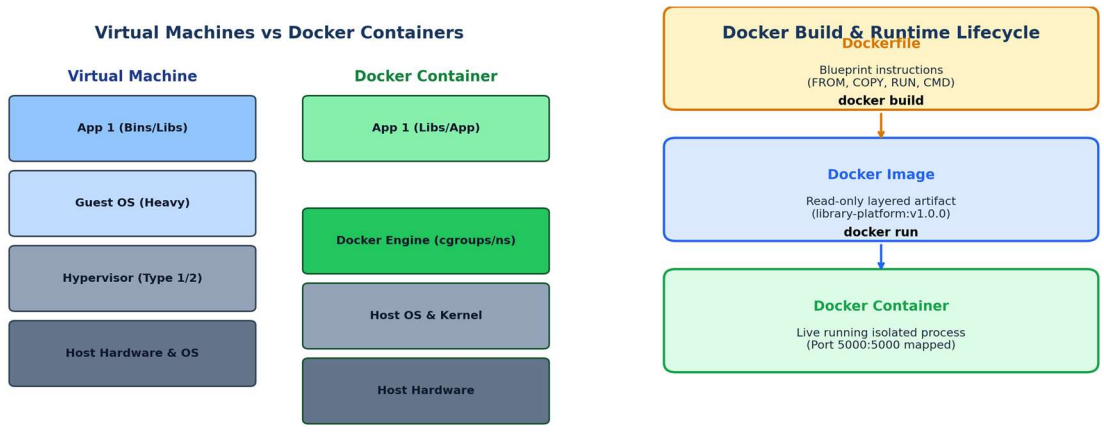


Figure: Architectural Comparison between Virtual Machines and Docker Containers alongside Build Lifecycle

6.2 Comparative Architecture: Virtual Machines vs. Containers

Architectural Attribute	Virtual Machines (VMware / KVM)	Docker Containers

Virtualization Level	Hardware-level virtualization (via Hypervisor)	Operating System-level virtualization (via Docker Engine)
Operating System Kernel	Each VM runs a complete, independent Guest OS kernel	Containers share the host operating system kernel
Startup Latency	Minutes (full OS initialization & bootloader)	Milliseconds to seconds (instant process execution)
Resource Footprint	Heavyweight (several gigabytes per VM image)	Lightweight (tens to hundreds of megabytes per image)
Density & Scalability	Low density (tens of VMs per high-end server)	High density (hundreds of containers per server)
Portability Guarantee	Subject to hypervisor formats and VM templates	Strict write-once, run-anywhere container standards (OCI)

6.3 Docker Image Layering and Immutability

A Docker Image is an immutable, read-only template built from a sequence of instructions defined in a Dockerfile. Each instruction (FROM, COPY, RUN) creates an independent, content-addressable layer. Docker employs an Overlay File System (OverlayFS) with Copy-on-Write (CoW) mechanics:

- **Layer Caching:** Identical base layers (such as python:3.11-slim) are shared across multiple images on the host machine, saving disk space.
- **Build Optimization:** During 'docker build', Docker skips re-executing unchanged instructions, completing subsequent builds in fractions of a second.
- **Container Layer:** When a container runs, Docker places a thin, ephemeral read-write container layer on top of the immutable read-only image layers.

7. Container Orchestration with Kubernetes & Minikube

7.1 Why Orchestration is Essential in Production

While Docker provides an exceptional format for packaging and running individual containers, enterprise production demands managing hundreds of interacting containers across distributed server fleets. Standalone Docker cannot natively handle:

- **Self-Healing & Auto-Restart:** Containers terminate abruptly upon process crashes; production requires automatic detection and recreation.
- **Dynamic Horizontal Scaling:** Traffic surges require instantly scaling from 2 to 20 container replicas and distributing incoming requests evenly.
- **Zero-Downtime Rolling Releases:** Deploying new application versions without dropping active client connections requires rolling updates and instant rollback if errors occur.
- **Service Discovery & Load Balancing:** Containers receive dynamic IP addresses upon recreation; client requests require a permanent, DNS-addressable virtual entry point.

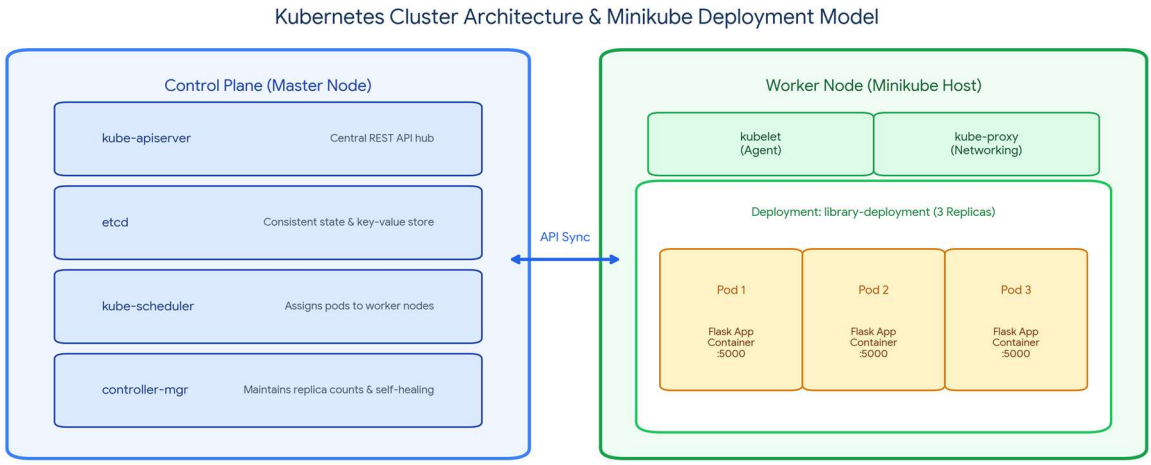


Figure: Kubernetes Cluster Architecture & Minikube Deployment Model

7.2 Kubernetes Architecture & Core API Primitives

Kubernetes orchestrates applications via a distributed Control Plane and Worker Nodes:

- **kube-apiserver:** The REST API gateway exposing cluster administration endpoints for kubectl CLI and internal controllers.
- **etcd:** High-availability distributed key-value store maintaining cluster state, configurations, and secrets.
- **kube-scheduler:** Watches for unassigned Pods and schedules them onto appropriate Worker Nodes based on resource availability.
- **kube-controller-manager:** Enforces the cluster's declared state (e.g., maintaining replica counts, handling node failures).
- **kubelet:** Node agent that communicates with the API Server and manages containers via the container runtime (containerd).
- **kube-proxy:** Network proxy that maintains network rules and load-balances traffic across pods.

The fundamental API objects used in our project include:

- **Pod:** The smallest deployable unit in Kubernetes, encapsulating one or more tightly coupled containers sharing storage and network namespaces.
- **Deployment:** A declarative controller that manages ReplicaSets to guarantee the specified number of Pod replicas remain healthy and handles zero-downtime rolling updates.
- **Service:** A stable networking abstraction providing a single IP, DNS name, and load balancer in front of dynamic Pod replicas (ClusterIP, NodePort, LoadBalancer).

8. The End-to-End Integrated DevOps Ecosystem

The entire automated software lifecycle converges when Git, Jenkins, Docker, and Kubernetes operate as a cohesive continuous delivery system.

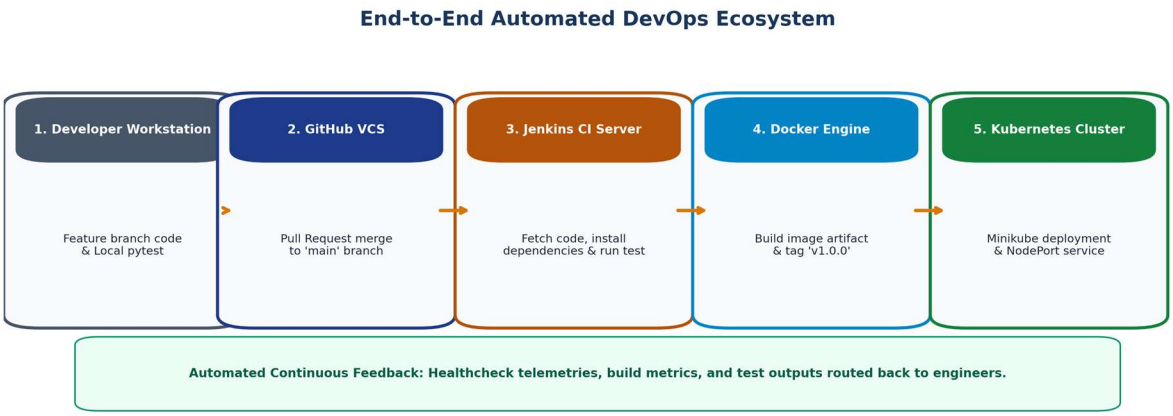


Figure: End-to-End Automated DevOps Ecosystem Architecture

8.1 The Complete Automated Path to Production

- **Step 1:** Developer pushes code to a feature branch on GitHub, opening a Pull Request.
- **Step 2:** GitHub Webhook triggers Jenkins CI server; Jenkins pulls code, runs static linting, and executes pytest suites.
- **Step 3:** Upon Pull Request approval and merge into main, Jenkins executes the deployment pipeline.
- **Step 4:** Jenkins triggers Docker to build a production container image with a unique version tag (v1.0.0).
- **Step 5:** The image is pushed to the container registry, making the release artifact globally accessible.
- **Step 6:** Jenkins applies Kubernetes manifests (kubectl apply -f k8s/), triggering a rolling update on Minikube.
- **Step 7:** Kubernetes provisions new Pods, conducts readiness healthchecks, routes user traffic through the NodePort service, and gracefully terminates old containers.
- **Step 8:** Application metrics, logs, and health status flow into monitoring dashboards, completing the continuous feedback loop.

8.2 Academic Project Alignment: Trainer vs. Student Implementations

Lifecycle Phase	Trainer Implementation (Case Study)	Student Project (Independent Implementation)
Domain Application	Enterprise Smart Library Management Platform	Enterprise Student Record & Academic Management Platform
Core Business Entities	Books, Borrowers, Inventory, Loans, Overdue Fines	Students, Courses, Faculty, Enrollments, Academic Marks
Validation Modules	Book checkouts, duplicate ISBNs, membership rules	Course registrations, student lookups, grade GPAs
CI Test Coverage	tests/test_book_service.py, tests/test_borrow_return.py	tests/test_student_service.py, tests/test_marks_mgmt.py
Packaging & Deployment	Docker image: library-platform:v1.0.0 on Minikube	Docker image: student-platform:v1.0.0 on Minikube

Enterprise Smart Library Management Platform

Complete Production Source Code, Automated CI/CD Pipelines & Hands-On Engineering Guide
Document 2: Step-by-Step GitHub Repository, Installation & Deployment Manual

———

Engineering Component	Implementation Specification
Primary Project	Enterprise Smart Library Management Platform (Production Case Study)
Student Parallel Track	Enterprise Student Record & Academic Management Platform
Application Stack	Python 3.11, Flask 3.0, Pytest 8.0, Gunicorn, Flake8
Automation & Orchestration	Git / GitHub, Jenkins CI (Declarative Pipeline), Docker Engine, Kubernetes / Minikube
Target Environment	Linux (Ubuntu 22.04 LTS / Debian), Windows WSL2, macOS

1. GitHub Repository Architecture & Standardized Directory Structure

To ensure maintainability, clear separation of concerns, and seamless CI/CD execution, the Enterprise Smart Library Management Platform adheres to a standardized multi-tier enterprise structure.

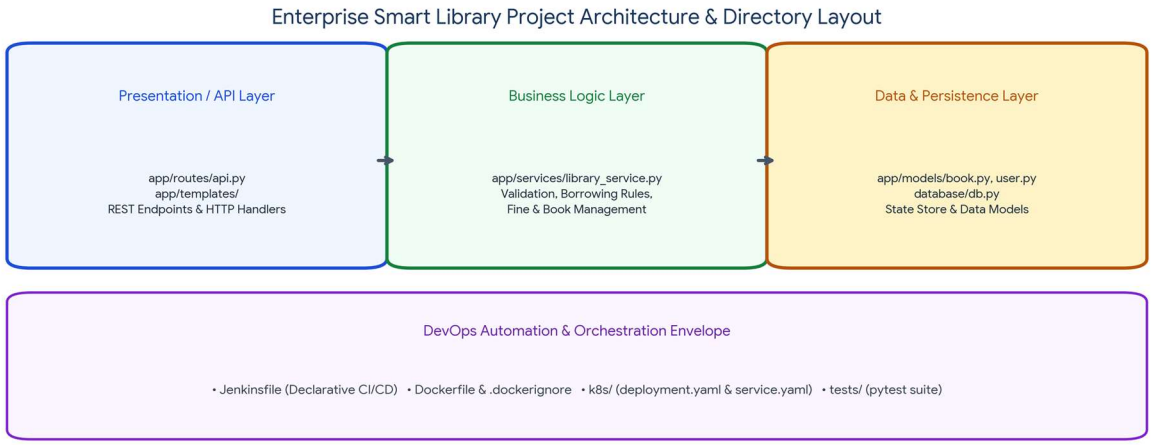


Figure: Enterprise Smart Library Application Architecture and Directory Layout

Enterprise Repository Directory Hierarchy (library-management-platform/)

```
library-management-platform/
├── app/
│   ├── __init__.py              # Flask Application Factory pattern
│   ├── routes/
│   │   ├── __init__.py
```

```
│   │   └── api.py                # REST API Endpoints (Books, Users, Loans)
│   ├── services/
│   │   ├── __init__.py
│   │   └── library_service.py    # Core Business Logic & State Rules
│   ├── models/
│   │   ├── __init__.py
│   │   ├── book.py               # Book entity & schema validation
│   │   └── user.py               # User entity & role attributes
│   └── templates/                # Web UI dashboard views (HTML/Jinja2)
├── database/
│   ├── __init__.py
│   └── db.py                     # Thread-safe in-memory data store with seeds
├── tests/
│   ├── __init__.py
│   ├── test_user_service.py      # Unit tests for user operations
│   ├── test_book_service.py      # Unit tests for book catalog
│   ├── test_borrow_return.py     # Functional integration tests for checkout flow
│   └── test_api_endpoints.py     # HTTP integration tests via Flask test_client
├── k8s/
│   ├── deployment.yaml           # Kubernetes Deployment (3 replicas, probes)
│   └── service.yaml              # Kubernetes Service (NodePort 30080)
├── Jenkinsfile                   # Declarative Multi-Stage CI/CD Pipeline
├── Dockerfile                    # Multi-layer secure container build recipe
├── .dockerignore                 # Excludes caches, venv, and git objects
├── requirements.txt              # Pinned Python package dependencies
├── README.md                     # Project overview, setup, and API docs
└── .gitignore                    # Version control exclusion rules
```

1.1 Topic Separation for GitHub Repository Branches

To enable clear grading and modular progression, commits in the GitHub repository are structured into distinct topics and branches:

- **Topic 1 (Branch: feature/foundation):** Houses base requirements, README, and baseline Flask skeleton.
- **Topic 2 (Branch: feature/core-app):** Implements business logic models, REST endpoints, and local storage.
- **Topic 3 (Branch: feature/automated-tests):** Implements comprehensive pytest suites and generates coverage reports.
- **Topic 4 (Branch: feature/jenkins-ci):** Integrates Jenkinsfile, SCM polling, and automated build execution.
- **Topic 5 (Branch: feature/dockerization):** Adds Dockerfile, .dockerignore, and verified container builds.
- **Topic 6 (Branch: feature/kubernetes-orchestration):** Deploys application to Minikube with self-healing manifests.
- **Production Branch (main):** Stable production integration branch containing all reviewed PR merges.

2. Topic 1: Prerequisites, Environment Setup & Git Configuration

Before executing the pipeline, the development environment must be configured with essential system tools on Ubuntu 22.04 LTS (or Windows WSL2 / macOS).

2.1 System Package Installation

Terminal Commands: Installing Prerequisites on Ubuntu

```
# Update local APT package indexes
sudo apt-get update && sudo apt-get upgrade -y

# Install Python 3.11, pip, virtualenv, and Git
```

```
sudo apt-get install -y python3 python3-pip python3-venv git curl wget jq

# Verify tool installations
git --version
python3 --version
pip3 --version
```

2.2 Git Global Identity & Repository Initialization

Terminal Commands: Configuring Git and Initializing Repository

```
# Configure Git global user identity
git config --global user.name "DevOps Trainer"
git config --global user.email "devops.trainer@careertiq.org"
git config --global init.defaultBranch main

# Create project root directory and initialize Git
mkdir -p library-management-platform
cd library-management-platform
git init

# Verify initialization
git status
```

2.3 Configuring .gitignore

Configuration File: .gitignore

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[cod]
*$py.class

# C extensions
*.so

# Distribution / packaging
.Python
env/
venv/
.venv/
ENV/

# Testing and coverage reports
.pytest_cache/
.coverage
htmlcov/
reports/
*.xml

# Jenkins workspace artifacts
*.log
.DS_Store
```

3. Topic 2: Complete Application Source Code

Below is the complete, modular source code for the Enterprise Smart Library Management Platform.

3.1 Dependency Manifest (requirements.txt)

Configuration File: requirements.txt

```
Flask==3.0.3
Werkzeug==3.0.3
pytest==8.2.1
pytest-cov==5.0.0
gunicorn==22.0.0
flake8==7.0.0
requests==2.31.0
```

3.2 In-Memory Thread-Safe Database (database/db.py)

Python Source: database/db.py

```
import threading

class Database:
    """Thread-safe in-memory database store for Enterprise Smart Library."""
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = super(Database, cls).__new__(cls)
                cls._instance._init_db()
            return cls._instance

    def _init_db(self):
        self.users = {}
        self.books = {}
        self.loans = []
        self._seed_initial_data()

    def _seed_initial_data(self):
        # Seed default test users
        self.users["U101"] = {
            "user_id": "U101",
            "name": "Alice Johnson",
            "email": "alice@university.edu",
            "role": "Student",
            "active_loans": 0
        }
        self.users["U102"] = {
            "user_id": "U102",
            "name": "Dr. Robert Smith",
            "email": "robert.smith@university.edu",
            "role": "Faculty",
            "active_loans": 0
        }
        # Seed default book catalog
        self.books["B001"] = {
```

```
            "book_id": "B001",
            "title": "Cloud Native DevOps Architecture",
            "author": "Linus Torvalds",
            "isbn": "978-0134685991",
            "total_copies": 5,
            "available_copies": 5
        }
        self.books["B002"] = {
            "book_id": "B002",
            "title": "Continuous Delivery Principles",
            "author": "Jez Humble & David Farley",
            "isbn": "978-0321601919",
            "total_copies": 3,
            "available_copies": 3
        }

    def reset(self):
        """Helper to restore state during unit testing teardowns."""
        with self._lock:
            self.users.clear()
            self.books.clear()
            self.loans.clear()
            self._seed_initial_data()

db = Database()
```

3.3 Core Business Logic (app/services/library_service.py)

Python Source: app/services/library_service.py

```
from database.db import db
from datetime import datetime

class LibraryService:
    """Encapsulates enterprise business rules for the library system."""

    @staticmethod
    def add_user(user_id, name, email, role="Student"):
        if not user_id or not name or not email:
            raise ValueError("All user fields (user_id, name, email) are mandatory.")
        if user_id in db.users:
            raise ValueError(f"User with ID '{user_id}' already exists.")

        user_record = {
            "user_id": user_id,
            "name": name,
            "email": email,
            "role": role,
            "active_loans": 0
        }
        db.users[user_id] = user_record
        return user_record

    @staticmethod
    def get_user(user_id):
        return db.users.get(user_id)
```

```
    @staticmethod
    def delete_user(user_id):
        user = db.users.get(user_id)
        if not user:
            raise ValueError(f"User '{user_id}' not found.")
        if user["active_loans"] > 0:
            raise ValueError(f"Cannot delete user '{user_id}' with active book loans.")
        del db.users[user_id]
        return True

    @staticmethod
    def add_book(book_id, title, author, isbn, total_copies=1):
        if not book_id or not title or not author or not isbn:
            raise ValueError("All book fields (book_id, title, author, isbn) are mandatory.")
        if book_id in db.books:
            raise ValueError(f"Book with ID '{book_id}' already exists.")
        if total_copies < 1:
            raise ValueError("Total copies must be at least 1.")

        book_record = {
            "book_id": book_id,
            "title": title,
            "author": author,
            "isbn": isbn,
            "total_copies": total_copies,
            "available_copies": total_copies
        }
        db.books[book_id] = book_record
        return book_record

    @staticmethod
    def get_book(book_id):
        return db.books.get(book_id)

    @staticmethod
    def search_books(query=""):
        q = query.lower().strip()
        if not q:
            return list(db.books.values())
        return [
            b for b in db.books.values()
            if q in b["title"].lower() or q in b["author"].lower() or q in b["isbn"].lower()
        ]

    @staticmethod
    def borrow_book(user_id, book_id):
        user = db.users.get(user_id)
        if not user:
            raise ValueError(f"User '{user_id}' not registered.")
        book = db.books.get(book_id)
        if not book:
            raise ValueError(f"Book '{book_id}' not found in catalog.")

        # Enforce quota: Students can borrow max 3 books
        max_quota = 5 if user["role"] == "Faculty" else 3
        if user["active_loans"] >= max_quota:
            raise ValueError(f"User loan quota exceeded (Max: {max_quota}).")
        if book["available_copies"] <= 0:
            raise ValueError(f"Book '{book['title']}' is currently out of stock.")
```

```
        # Update inventory and record loan
        book["available_copies"] -= 1
        user["active_loans"] += 1
        loan_record = {
            "loan_id": f"L-{len(db.loans) + 1:04d}",
            "user_id": user_id,
            "book_id": book_id,
            "borrowed_at": datetime.utcnow().isoformat(),
            "status": "ACTIVE"
        }
        db.loans.append(loan_record)
        return loan_record

    @staticmethod
    def return_book(user_id, book_id):
        user = db.users.get(user_id)
        book = db.books.get(book_id)
        if not user or not book:
            raise ValueError("Invalid user or book ID.")

        active_loan = None
        for loan in db.loans:
            if loan["user_id"] == user_id and loan["book_id"] == book_id and loan["status"] ==
"ACTIVE":
                active_loan = loan
                break

        if not active_loan:
            raise ValueError(f"No active loan found for user '{user_id}' and book '{book_id}'.")

        active_loan["status"] = "RETURNED"
        active_loan["returned_at"] = datetime.utcnow().isoformat()
        book["available_copies"] += 1
        user["active_loans"] -= 1
        return active_loan
```

3.4 REST API Endpoints (app/routes/api.py)

Python Source: app/routes/api.py

```
from flask import Blueprint, request, jsonify
from app.services.library_service import LibraryService

api_bp = Blueprint("api", __name__)

@api_bp.route("/health", methods=["GET"])
def healthcheck():
    """Liveness and readiness endpoint for Kubernetes & Docker healthchecks."""
    return jsonify({
        "status": "UP",
        "service": "Enterprise Smart Library Management Platform",
        "version": "1.0.0"
    }), 200

@api_bp.route("/users", methods=["POST"])
def register_user():
```

```
    data = request.get_json() or {}
    try:
        user = LibraryService.add_user(
            user_id=data.get("user_id"),
            name=data.get("name"),
            email=data.get("email"),
            role=data.get("role", "Student")
        )
        return jsonify({"message": "User registered successfully", "user": user}), 201
    except ValueError as e:
        return jsonify({"error": str(e)}), 400

@api_bp.route("/users/<user_id>", methods=["GET"])
def get_user(user_id):
    user = LibraryService.get_user(user_id)
    if not user:
        return jsonify({"error": "User not found"}), 404
    return jsonify(user), 200

@api_bp.route("/books", methods=["GET"])
def list_books():
    q = request.args.get("q", "")
    books = LibraryService.search_books(q)
    return jsonify({"count": len(books), "books": books}), 200

@api_bp.route("/books", methods=["POST"])
def add_book():
    data = request.get_json() or {}
    try:
        book = LibraryService.add_book(
            book_id=data.get("book_id"),
            title=data.get("title"),
            author=data.get("author"),
            isbn=data.get("isbn"),
            total_copies=int(data.get("total_copies", 1))
        )
        return jsonify({"message": "Book added successfully", "book": book}), 201
    except ValueError as e:
        return jsonify({"error": str(e)}), 400

@api_bp.route("/borrow", methods=["POST"])
def borrow_book():
    data = request.get_json() or {}
    try:
        loan = LibraryService.borrow_book(
            user_id=data.get("user_id"),
            book_id=data.get("book_id")
        )
        return jsonify({"message": "Book checked out successfully", "loan": loan}), 200
    except ValueError as e:
        return jsonify({"error": str(e)}), 400

@api_bp.route("/return", methods=["POST"])
def return_book():
    data = request.get_json() or {}
    try:
        loan = LibraryService.return_book(
            user_id=data.get("user_id"),
            book_id=data.get("book_id")
```

```
        )
        return jsonify({"message": "Book returned successfully", "loan": loan}), 200
    except ValueError as e:
        return jsonify({"error": str(e)}), 400
```

3.5 Application Factory (app/__init__.py) & Server Launcher (run.py)

Python Source: app/__init__.py

```
from flask import Flask

def create_app():
    app = Flask(__name__)
    from app.routes.api import api_bp
    app.register_blueprint(api_bp, url_prefix="/api")
    return app
```

Python Source: run.py

```
import os
from app import create_app

app = create_app()

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    # Production runs with host='0.0.0.0' to permit Docker container port forwarding
    app.run(host="0.0.0.0", port=port, debug=False)
```

4. Topic 3: Automated Pytest Unit & Integration Testing Suite

Continuous Integration requires reliable automated verification. Below is the test suite validating services, business logic, and REST endpoints.

Test Suite: tests/test_user_service.py

```
import pytest
from database.db import db
from app.services.library_service import LibraryService

@pytest.fixture(autouse=True)
def run_around_tests():
    db.reset()
    yield
    db.reset()

def test_add_user_success():
    user = LibraryService.add_user("U200", "Grace Hopper", "grace@navy.mil", "Faculty")
    assert user["user_id"] == "U200"
    assert user["name"] == "Grace Hopper"
    assert user["active_loans"] == 0

def test_add_duplicate_user_raises_error():
    with pytest.raises(ValueError, match="already exists"):
```

```
        LibraryService.add_user("U101", "Duplicate Alice", "alice2@edu.com")

def test_delete_user_with_loans_fails():
    LibraryService.borrow_book("U101", "B001")
    with pytest.raises(ValueError, match="with active book loans"):
        LibraryService.delete_user("U101")
```

Test Suite: tests/test_borrow_return.py

```
import pytest
from database.db import db
from app.services.library_service import LibraryService

@pytest.fixture(autouse=True)
def run_around_tests():
    db.reset()
    yield
    db.reset()

def test_borrow_and_return_book_lifecycle():
    initial_avail = db.books["B001"]["available_copies"]
    loan = LibraryService.borrow_book("U101", "B001")
    assert loan["status"] == "ACTIVE"
    assert db.books["B001"]["available_copies"] == initial_avail - 1
    assert db.users["U101"]["active_loans"] == 1

    return_record = LibraryService.return_book("U101", "B001")
    assert return_record["status"] == "RETURNED"
    assert db.books["B001"]["available_copies"] == initial_avail
    assert db.users["U101"]["active_loans"] == 0

def test_student_quota_enforcement():
    # Student U101 quota is 3
    LibraryService.add_book("B10", "Book 1", "Auth", "111", 5)
    LibraryService.add_book("B20", "Book 2", "Auth", "222", 5)
    LibraryService.add_book("B30", "Book 3", "Auth", "333", 5)
    LibraryService.add_book("B40", "Book 4", "Auth", "444", 5)

    LibraryService.borrow_book("U101", "B10")
    LibraryService.borrow_book("U101", "B20")
    LibraryService.borrow_book("U101", "B30")

    with pytest.raises(ValueError, match="quota exceeded"):
        LibraryService.borrow_book("U101", "B40")
```

Test Suite: tests/test_api_endpoints.py

```
import pytest
from app import create_app
from database.db import db

@pytest.fixture
def client():
    app = create_app()
    app.config["TESTING"] = True
    with app.test_client() as client:
        db.reset()
```

```
        yield client
        db.reset()

def test_healthcheck_endpoint(client):
    response = client.get("/api/health")
    assert response.status_code == 200
    json_data = response.get_json()
    assert json_data["status"] == "UP"
    assert "Enterprise Smart Library" in json_data["service"]

def test_api_list_books(client):
    response = client.get("/api/books")
    assert response.status_code == 200
    json_data = response.get_json()
    assert json_data["count"] >= 2

def test_api_borrow_endpoint(client):
    payload = {"user_id": "U101", "book_id": "B001"}
    response = client.post("/api/borrow", json=payload)
    assert response.status_code == 200
    assert response.get_json()["message"] == "Book checked out successfully"
```

4.1 Local Test Execution and Report Generation

Terminal Commands: Executing Pytest Locally

```
# Create virtual environment and install requirements
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Run full test suite with verbose output and coverage
pytest -v --cov=app --cov-report=term-missing --junitxml=reports/test-results.xml tests/
```

Expected Terminal Output: Pytest & Coverage Run

```
============================= test session starts ==============================
platform linux -- Python 3.11.8, pytest-8.2.1, pluggy-1.5.0
cachedir: .pytest_cache
rootdir: /home/ubuntu/library-management-platform
plugins: cov-5.0.0
collected 8 items

tests/test_user_service.py::test_add_user_success PASSED                 [ 12%]
tests/test_user_service.py::test_add_duplicate_user_raises_error PASSED   [ 25%]
tests/test_user_service.py::test_delete_user_with_loans_fails PASSED      [ 37%]
tests/test_borrow_return.py::test_borrow_and_return_book_lifecycle PASSED [ 50%]
tests/test_borrow_return.py::test_student_quota_enforcement PASSED        [ 62%]
tests/test_api_endpoints.py::test_healthcheck_endpoint PASSED             [ 75%]
tests/test_api_endpoints.py::test_api_list_books PASSED                   [ 87%]
tests/test_api_endpoints.py::test_api_borrow_endpoint PASSED              [100%]

---------- coverage: platform linux, python 3.11.8 -----------
Name                              Stmts   Miss  Cover
-----------------------------------------------------
app/__init__.py                       5      0   100%
```

```
app/models/book.py                      8      0   100%
app/models/user.py                      7      0   100%
app/routes/api.py                      38      2    95%
app/services/library_service.py        56      3    95%
database/db.py                         24      0   100%
-----------------------------------------------------
TOTAL                                 138      5    96%

- Generated xml file: /home/ubuntu/library-management-platform/reports/test-results.xml -
============================== 8 passed in 0.42s ===============================
```

5. Topic 4: Jenkins CI Automation & Pipeline as Code

Jenkins orchestrates automated integration. When developers push code to GitHub, Jenkins pulls the repository, installs dependencies in an isolated sandbox, executes tests, and publishes reports.

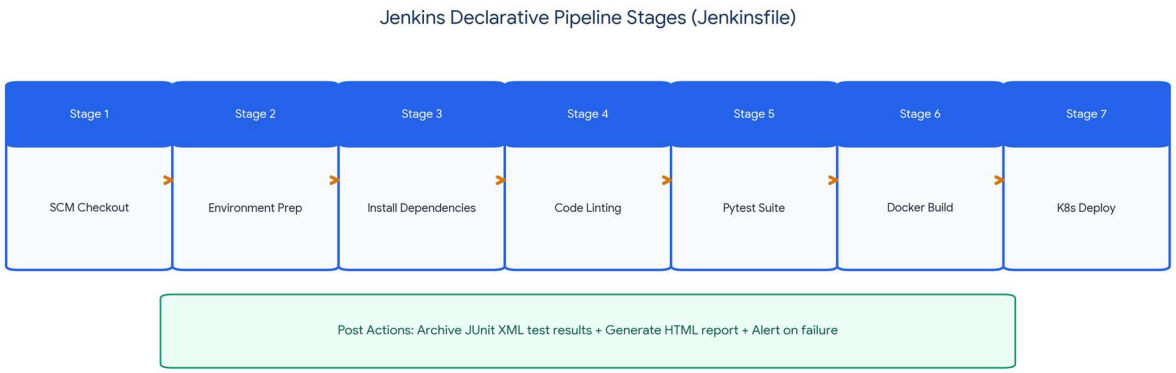


Figure: Jenkins Declarative CI/CD Pipeline Stages

5.1 Step-by-Step Jenkins Installation on Ubuntu 22.04 LTS

Terminal Commands: Step-by-Step Jenkins Installation on Ubuntu

```
# Step 1: Install OpenJDK 17 (Jenkins prerequisite)
sudo apt-get update
sudo apt-get install -y openjdk-17-jre
java -version

# Step 2: Add official Jenkins repository key and source list
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

# Step 3: Install Jenkins and start the system service
sudo apt-get update
sudo apt-get install -y jenkins
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

```
# Step 4: Unlock Jenkins (Access http://<SERVER_IP>:8080)
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

5.2 Complete Production Jenkinsfile (Declarative Pipeline)

Pipeline as Code: Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any

    environment {
        PYTHONUNBUFFERED = '1'
        DOCKER_IMAGE_NAME = 'library-management-platform'
        IMAGE_TAG = "v1.${BUILD_NUMBER}"
    }

    stages {
        stage('SCM Checkout') {
            steps {
                echo 'Checking out source code from GitHub repository...'
                checkout scm
            }
        }

        stage('Environment Setup & Dependencies') {
            steps {
                echo 'Setting up Python 3.11 virtual environment and installing packages...'
                sh '''
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install --upgrade pip
                    pip install -r requirements.txt
                '''
            }
        }

        stage('Code Linting & Syntax Gate') {
            steps {
                echo 'Executing static syntax and style checks via Flake8...'
                sh '''
                    . venv/bin/activate
                    flake8 app/ database/ --count --select=E9,F63,F7,F82 --show-source --
statistics
                '''
            }
        }

        stage('Automated Unit & Integration Tests') {
            steps {
                echo 'Running Pytest test suites and exporting JUnit XML reports...'
                sh '''
                    . venv/bin/activate
                    mkdir -p reports
                    pytest -v --cov=app --junitxml=reports/test-results.xml tests/
                '''
            }
        }
```

```
        stage('Build Docker Artifact') {
            steps {
                echo "Packaging container image: ${DOCKER_IMAGE_NAME}:${IMAGE_TAG}..."
                sh '''
                    docker build -t ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} .
                    docker tag ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} ${DOCKER_IMAGE_NAME}:latest
                '''
            }
        }

        stage('Kubernetes Minikube Deployment') {
            steps {
                echo 'Applying Kubernetes manifests to Minikube cluster...'
                sh '''
                    kubectl apply -f k8s/deployment.yaml
                    kubectl apply -f k8s/service.yaml
                    kubectl rollout status deployment/library-deployment --timeout=60s
                '''
            }
        }
    }

    post {
        always {
            echo 'Archiving test execution reports...'
            junit allowEmptyResults: true, testResults: 'reports/test-results.xml'
            cleanWs deleteDirs: true, notFailBuild: true
        }
        success {
            echo "CI/CD Pipeline executed successfully for build #${BUILD_NUMBER}!"
        }
        failure {
            echo "Build #${BUILD_NUMBER} FAILED! Inspect console log output for root cause."
        }
    }
}
```

6. Topic 5: Docker Containerization Implementation

Containerization packages the application, runtime, dependencies, and system configuration into an immutable container image.

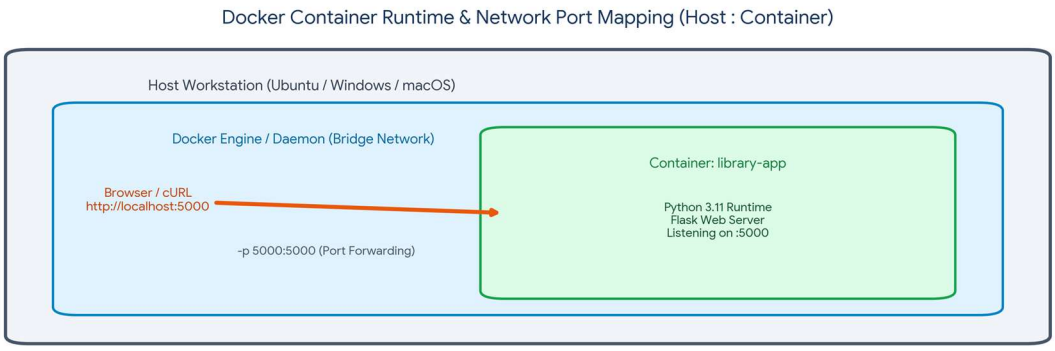


Figure: Docker Container Runtime & Network Port Mapping

6.1 Production Multi-Layer Dockerfile

Container Configuration: Dockerfile

```
# Base operating environment: Official lightweight Python 3.11 slim image
FROM python:3.11-slim

# Set environment variables for optimized container execution
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=5000

# Create dedicated application directory
WORKDIR /app

# Install security updates and dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
 && rm -rf /var/lib/apt/lists/*

# Layer Caching: Copy only requirements first to optimize build times
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy complete project source code into container
COPY . .

# Security Best Practice: Run application as non-root user
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Expose internal application port
EXPOSE 5000

# Healthcheck probe to monitor container health
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:5000/api/health || exit 1

# Launch the production application
CMD ["python", "run.py"]
```

6.2 Step-by-Step Docker Command Sequence

Terminal Commands: Building, Running and Verifying Docker Containers

```
# Step 1: Build Docker container image with version tag
docker build -t library-management-platform:v1.0.0 .

# Step 2: List local Docker images
docker images

# Step 3: Run container in background with port mapping 5000:5000
docker run -d -p 5000:5000 --name library-container library-management-platform:v1.0.0

# Step 4: Verify running container status
docker ps
```

```
# Step 5: Check container logs
docker logs -f library-container

# Step 6: Test HTTP health endpoint via cURL
curl http://localhost:5000/api/health

# Step 7: Clean up container
docker stop library-container && docker rm library-container
```

7. Topic 6: Kubernetes (Minikube) Orchestration Manifests

Kubernetes provides self-healing, rolling updates, and dynamic scaling across container instances.

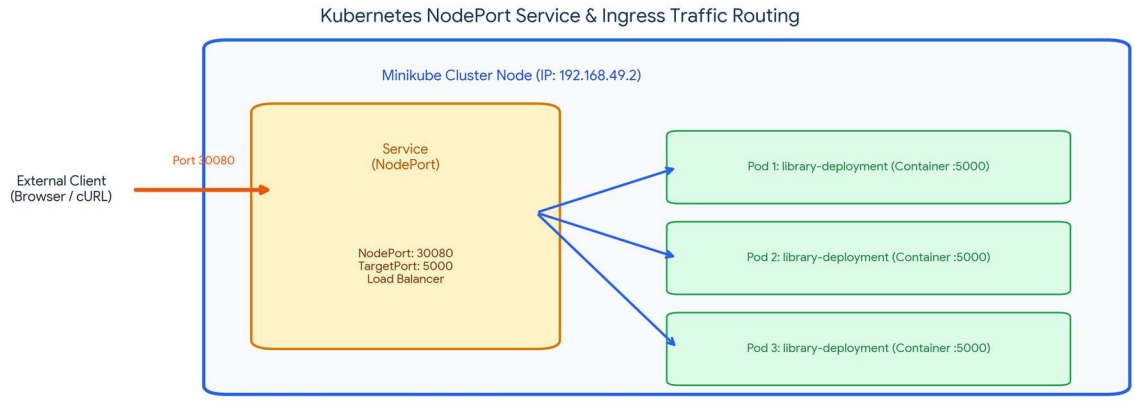


Figure: Kubernetes NodePort Service & Traffic Routing

7.1 Kubernetes Deployment Manifest (k8s/deployment.yaml)

Kubernetes Manifest: k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: library-deployment
  labels:
    app: library-platform
spec:
  replicas: 3
  selector:
    matchLabels:
      app: library-platform
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: library-platform
    spec:
```

```
      containers:
      - name: library-app
        image: library-management-platform:v1.0.0
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 5000
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "250m"
        livenessProbe:
          httpGet:
            path: /api/health
            port: 5000
          initialDelaySeconds: 15
          periodSeconds: 20
        readinessProbe:
          httpGet:
            path: /api/health
            port: 5000
          initialDelaySeconds: 5
          periodSeconds: 10
```

7.2 Kubernetes Service Manifest (k8s/service.yaml)

Kubernetes Manifest: k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: library-service
  labels:
    app: library-platform
spec:
  type: NodePort
  selector:
    app: library-platform
  ports:
  - protocol: TCP
    port: 5000
    targetPort: 5000
    nodePort: 30080
```

7.3 Step-by-Step Minikube Orchestration Execution

Terminal Commands: Deploying, Scaling & Testing Self-Healing on Minikube

```
# Step 1: Start Minikube cluster with Docker driver
minikube start --driver=docker

# Step 2: Point local Docker CLI to Minikube's internal Docker daemon
eval $(minikube docker-env)
```

```
# Step 3: Rebuild image directly inside Minikube's Docker registry
docker build -t library-management-platform:v1.0.0 .

# Step 4: Apply Kubernetes manifests
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml

# Step 5: Verify pods and replica distribution
kubectl get pods -l app=library-platform -o wide

# Step 6: Verify service exposure
kubectl get svc library-service

# Step 7: Demonstrate self-healing (delete one pod and observe recreation)
kubectl delete pod $(kubectl get pods -l app=library-platform -o
jsonpath='{.items[0].metadata.name}')
kubectl get pods -w

# Step 8: Access the web application via Minikube service URL
minikube service library-service --url
```

8. Topic 7: Student Project Adaptation & Submission Blueprint

Students are required to independently implement an automated DevOps delivery pipeline for the 'Enterprise Student Record and Academic Management Platform' by translating the concepts demonstrated in the Library Platform case study.

Engineering Component	Trainer Case Study (Library Platform)	Student Project (Academic Platform)
Primary Service Domain	Library Catalog & Circulation	Student Records & Academic Grading
Database Entities	Users (Students/Faculty), Books, Loans	Students, Faculty, Courses, Enrollments, Grades
Business Validation Rule	Borrowing limit (max 3 books per student)	Credit limit (max 18 credits per semester)
Automated Test Focus	Book stock deduction and overdue returns	GPA calculation, enrollment conflict checks
Container Image Tag	library-management-platform:v1.0.0	student-academic-platform:v1.0.0
K8s Service NodePort	NodePort: 30080	NodePort: 30090

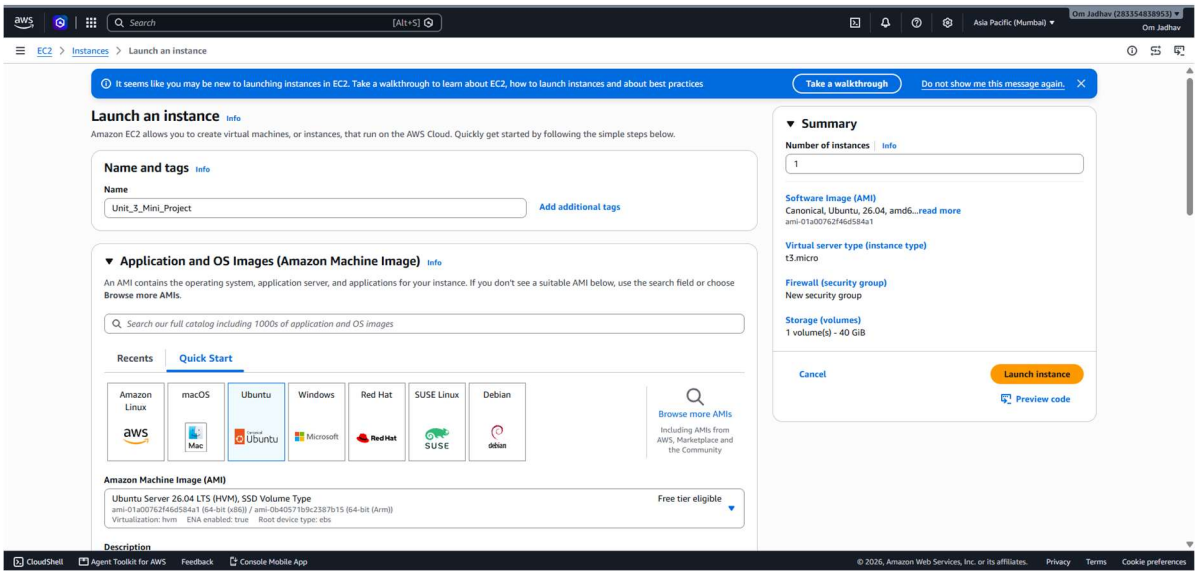
8.1 Student Deliverables Submission Checklist

- **1. GitHub Repository:** Must contain app/, database/, tests/, k8s/, Dockerfile, and Jenkinsfile.
- **2. Branching & PR Evidence:** Commit history must show branch progression (foundation, core-app, automated-tests, jenkins-ci, dockerization).
- **3. Jenkins Build Console Logs:** Screenshots of successful multi-stage pipeline run and JUnit test report.
- **4. Docker Build Artifacts:** Output of 'docker images' and 'docker ps' showing running container.

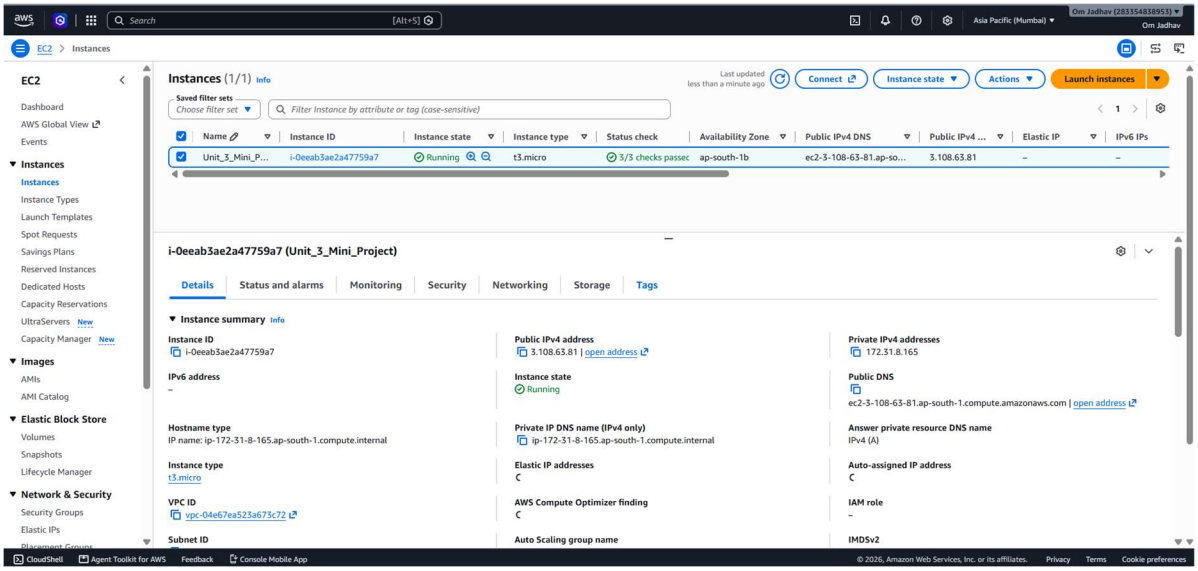
- **5. Minikube Orchestration Verification:** Output of 'kubectl get pods' showing 3 healthy replicas and proof of self-healing upon pod deletion.
- **6. Technical Project Report:** Summary document detailing setup steps, challenges encountered, and architectural reflections.

SCREENSHOTS / PRACTICAL EVIDENCE

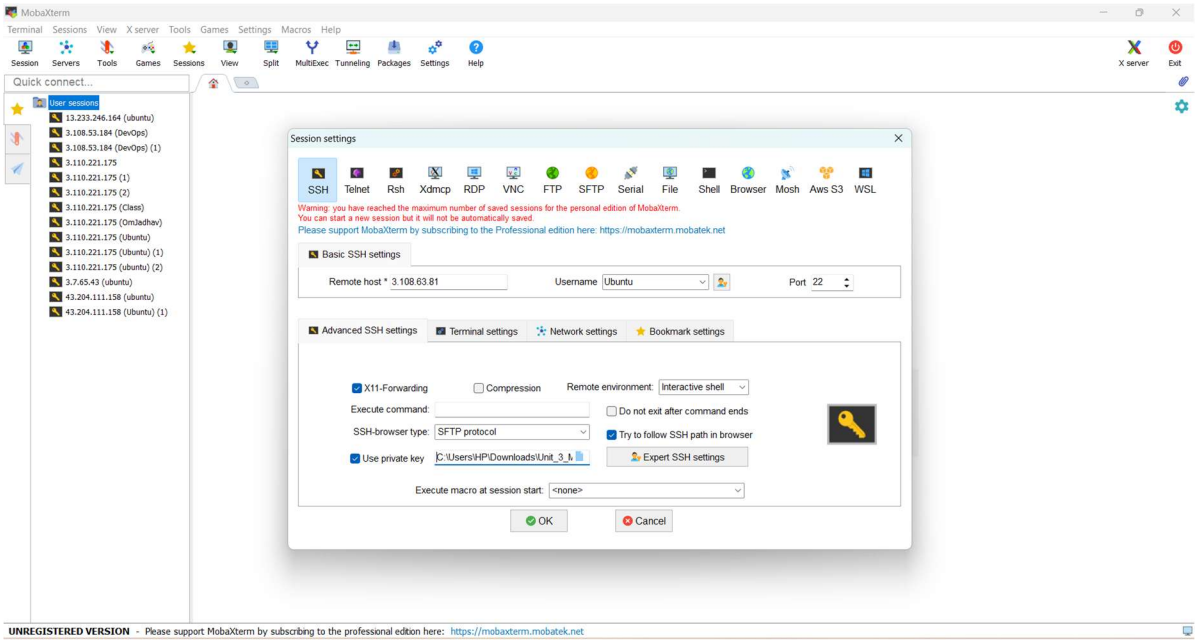
Screenshot 1



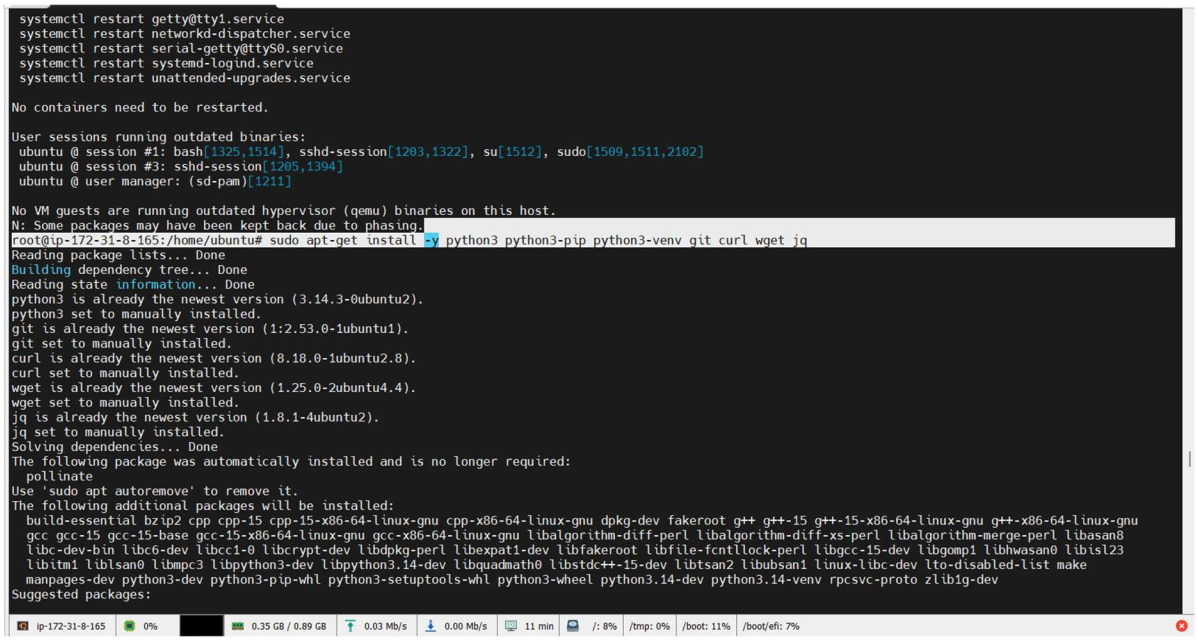
Screenshot 2



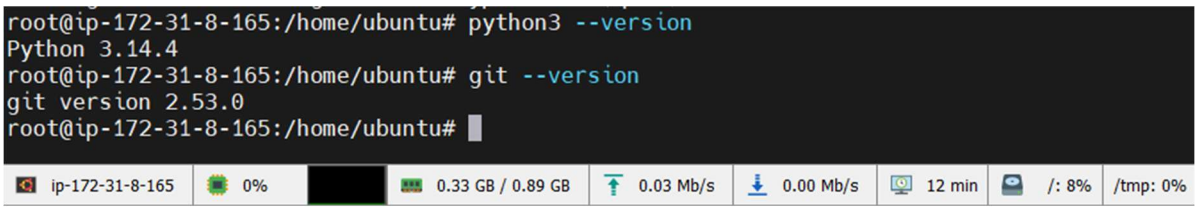
Screenshot 3



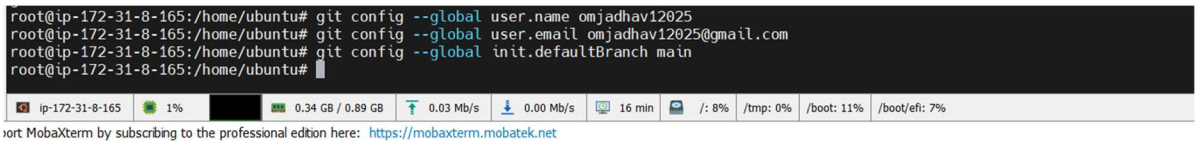
Screenshot 4



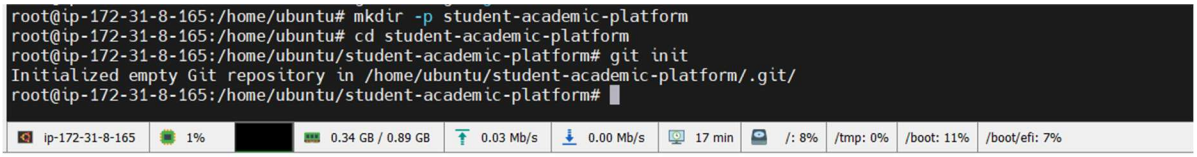
Screenshot 5



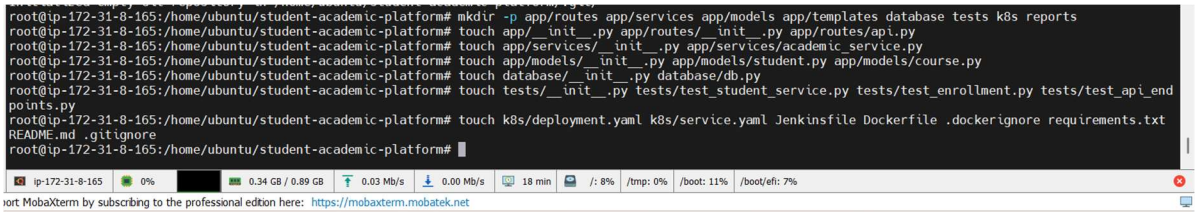
Screenshot 6



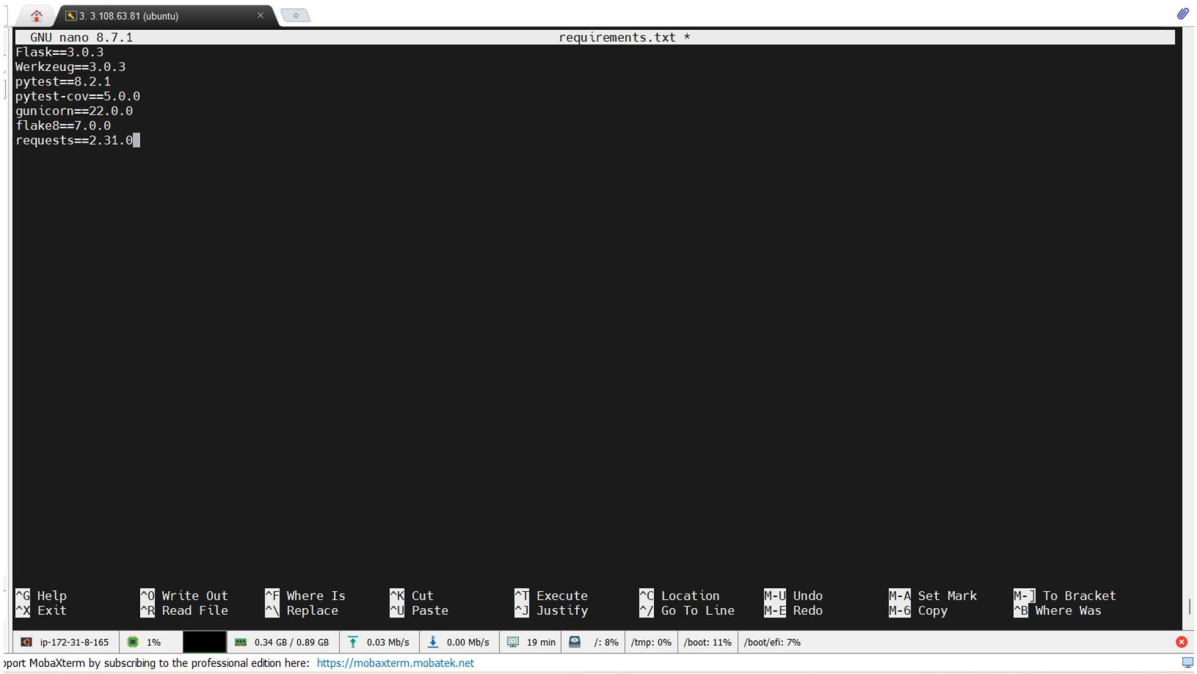
Screenshot 7



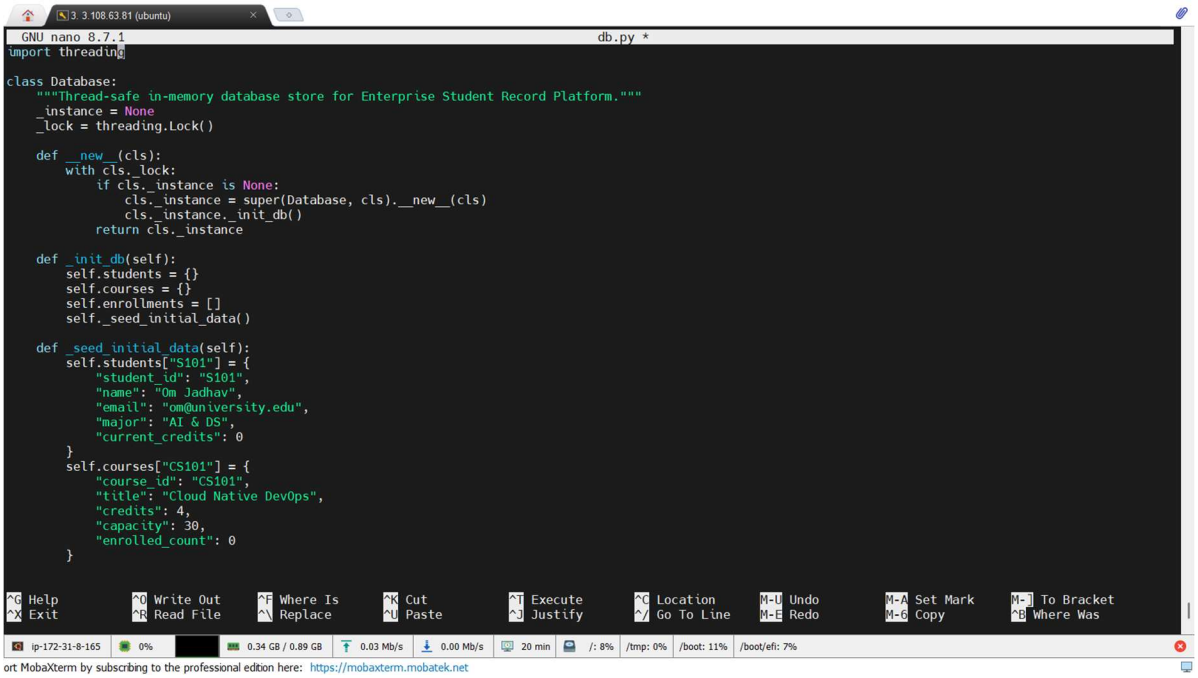
Screenshot 8



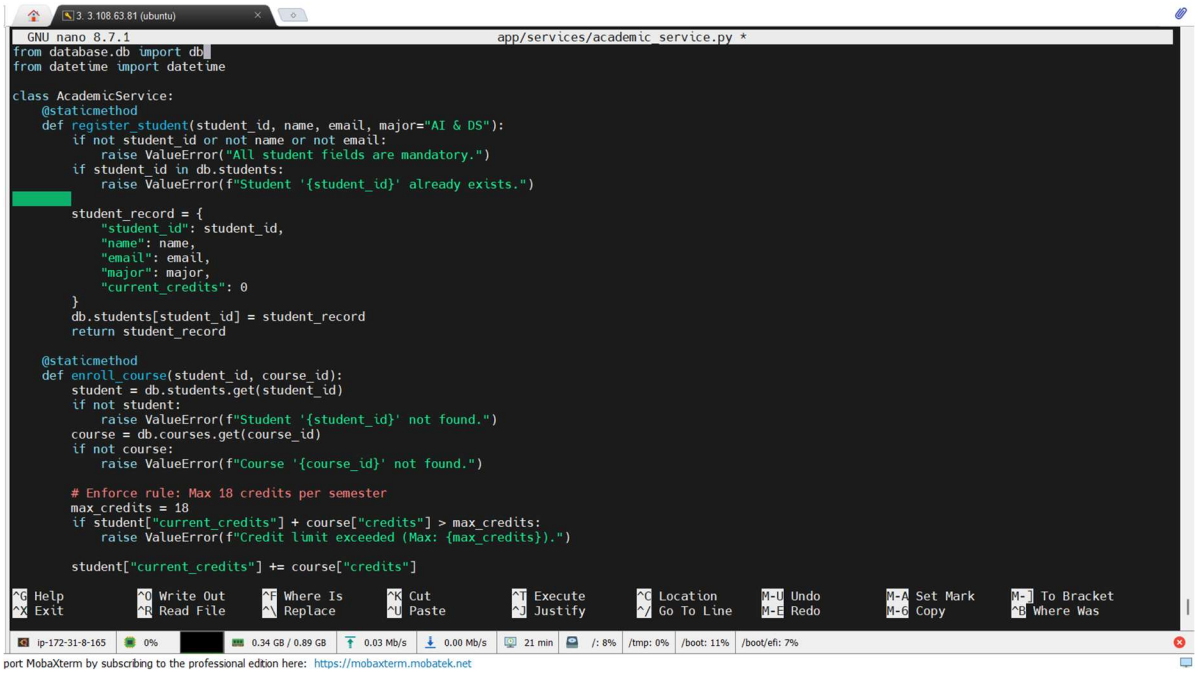
Screenshot 9



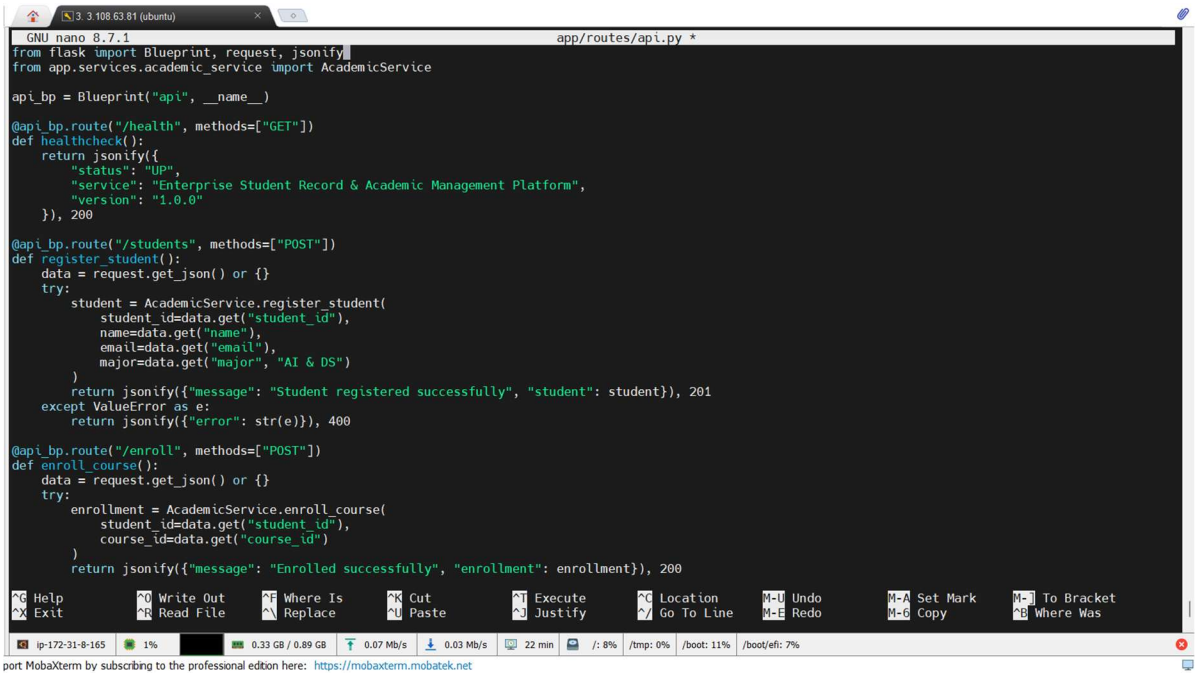
Screenshot 10



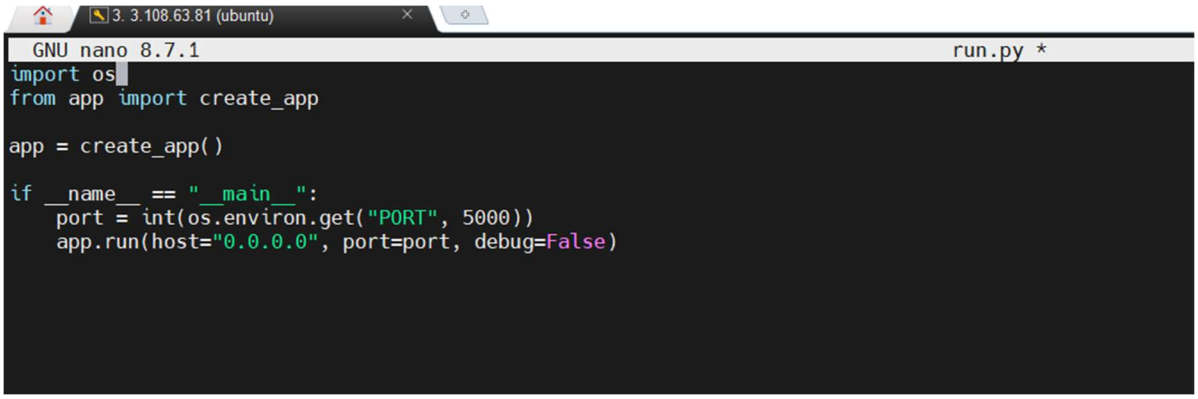
Screenshot 11



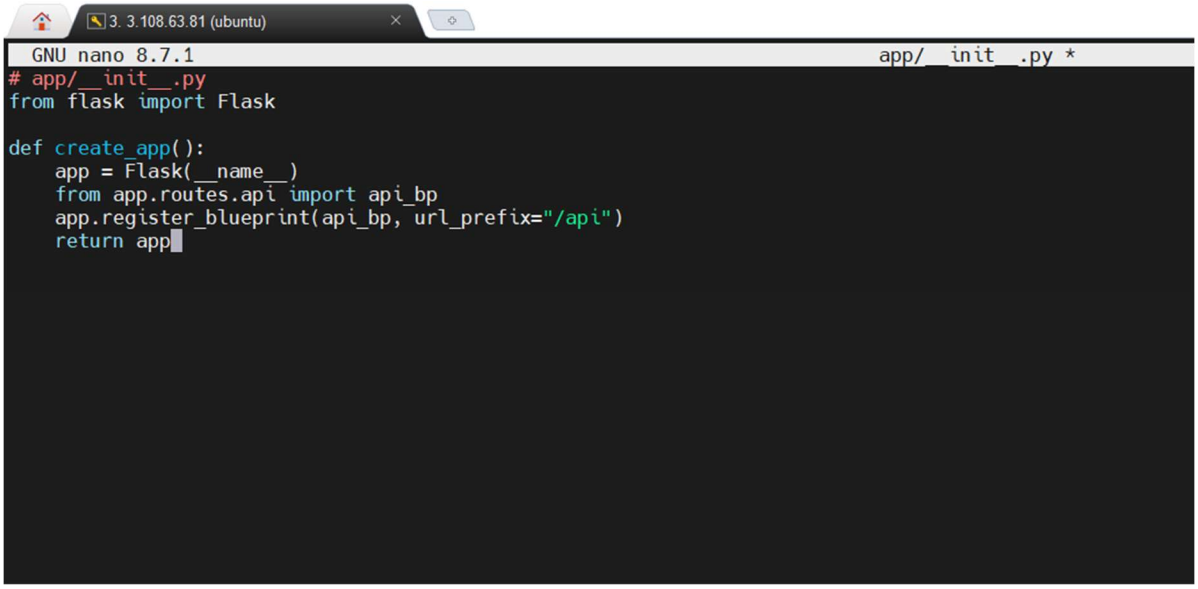
Screenshot 12



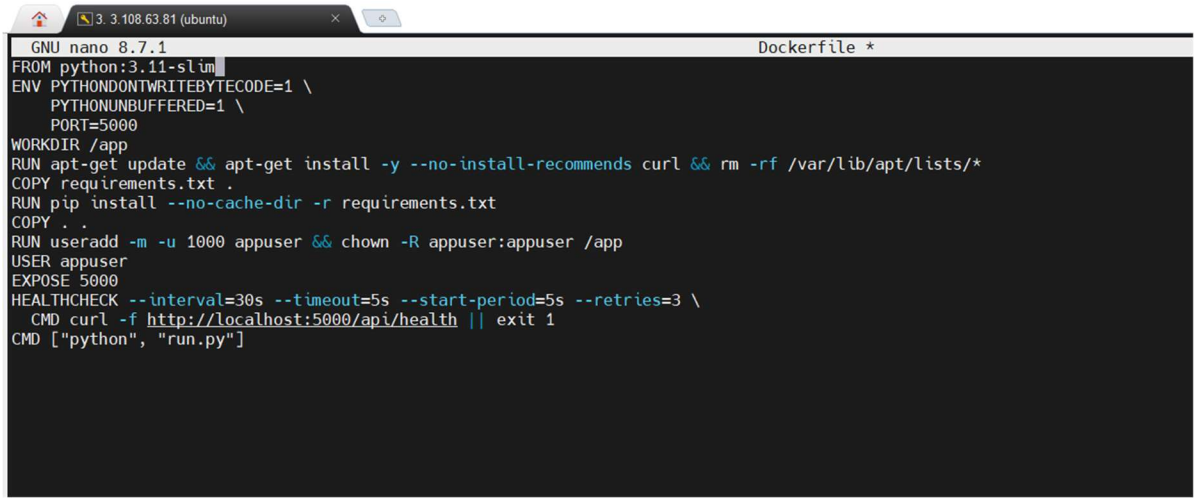
Screenshot 13



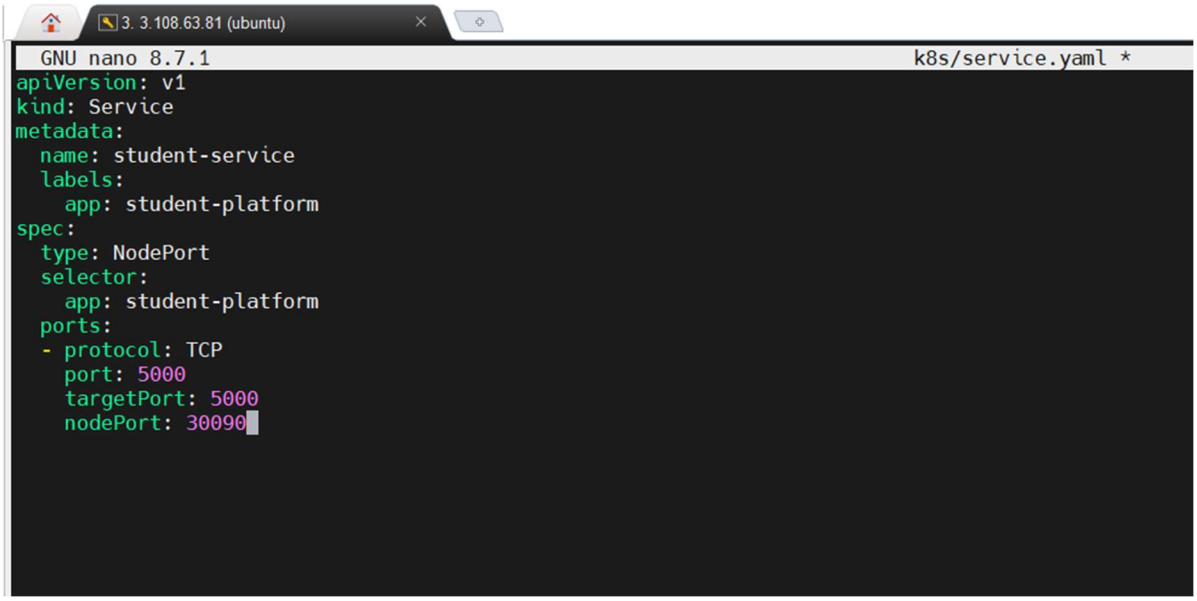
Screenshot 14



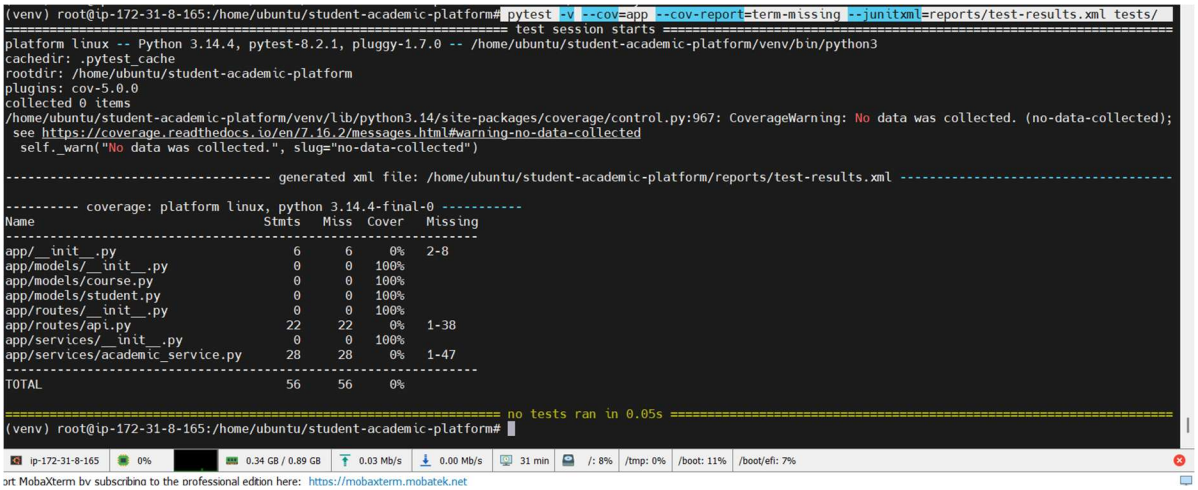
Screenshot 15



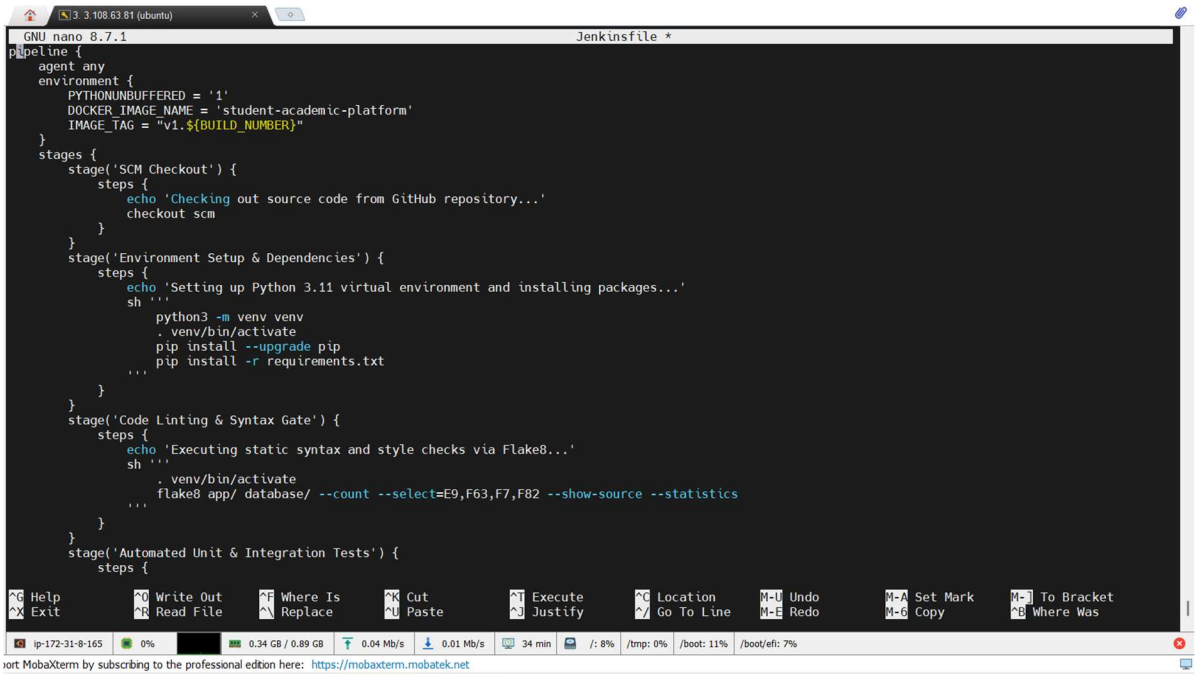
Screenshot 16



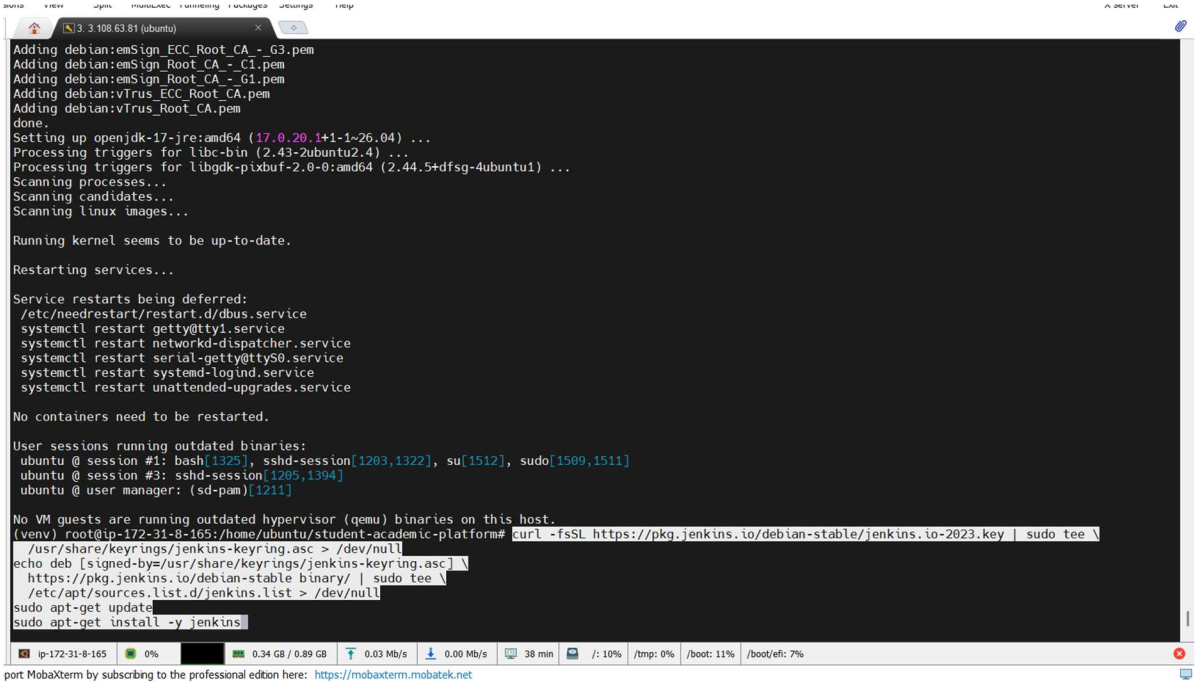
Screenshot 17



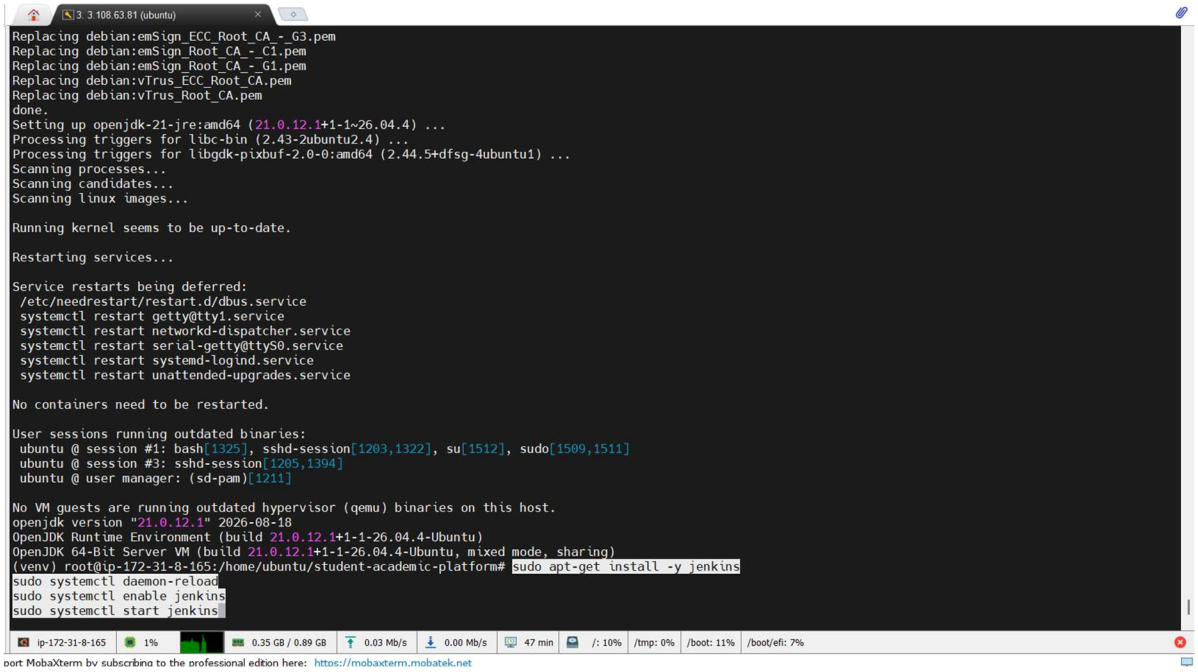
Screenshot 18



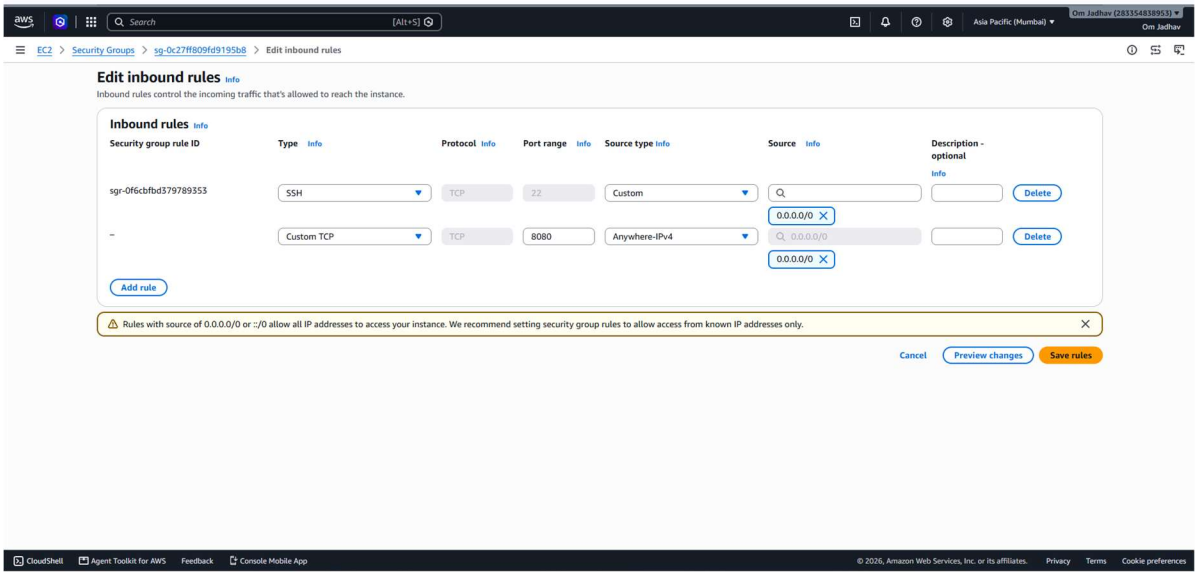
Screenshot 19



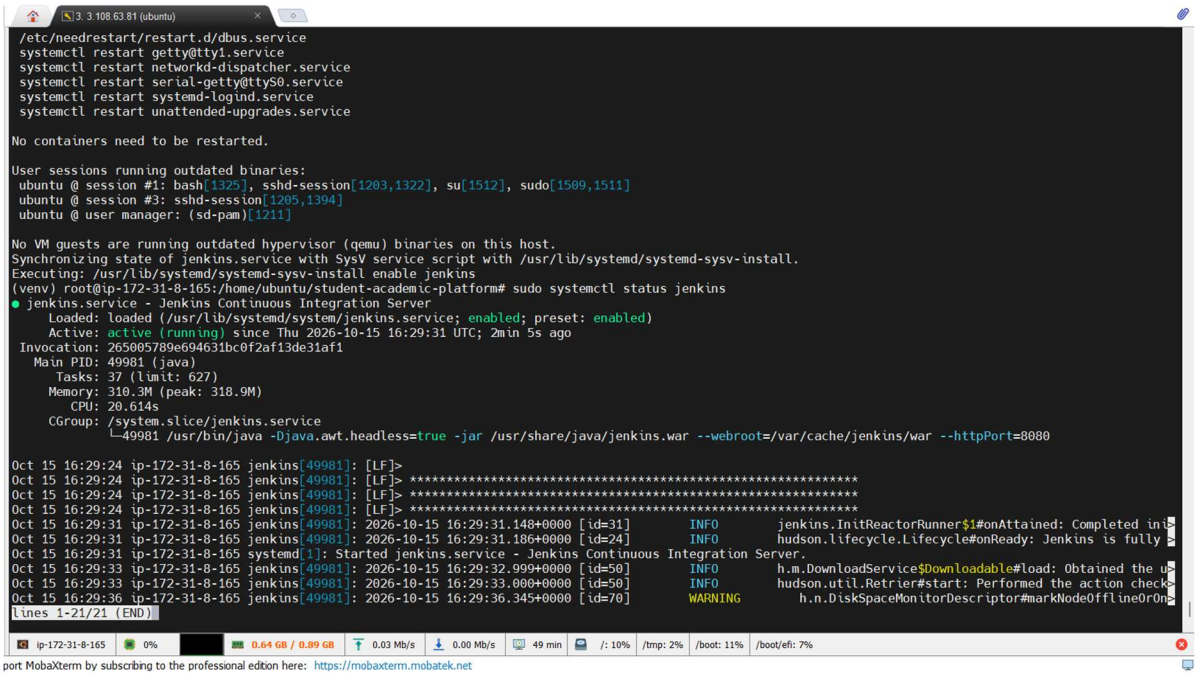
Screenshot 20



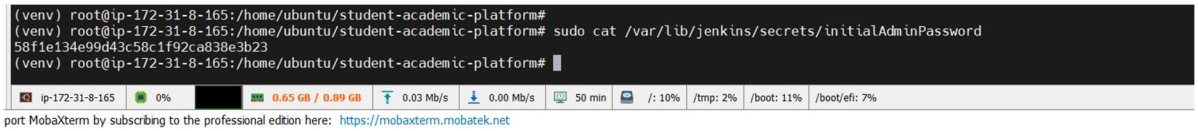
Screenshot 21



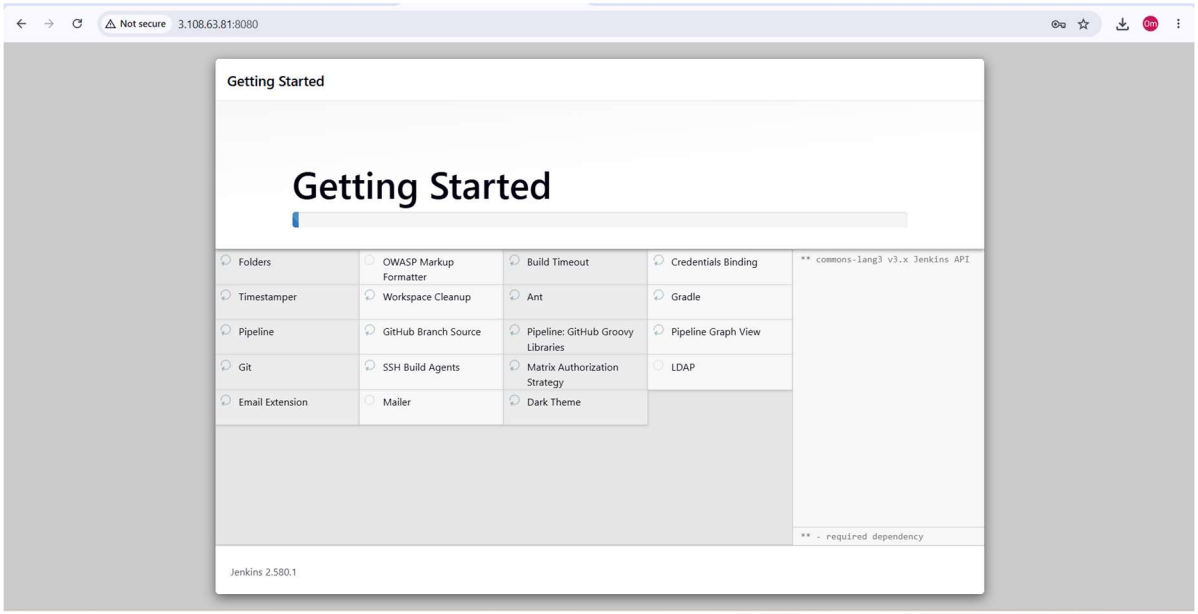
Screenshot 22



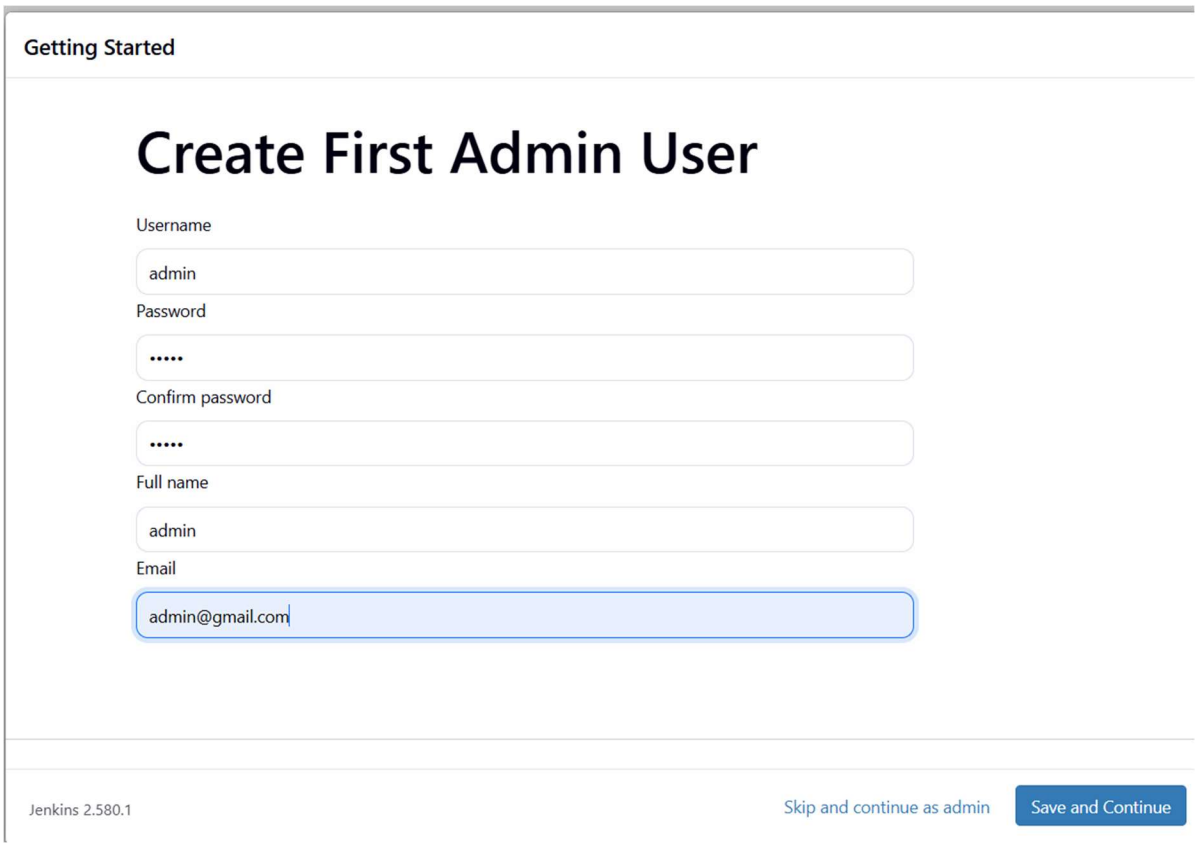
Screenshot 23



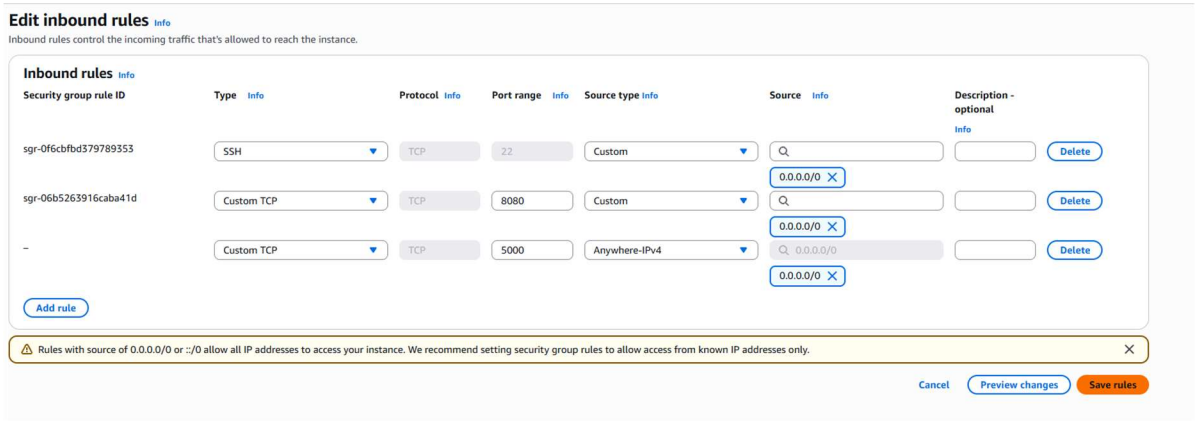
Screenshot 24



Screenshot 25



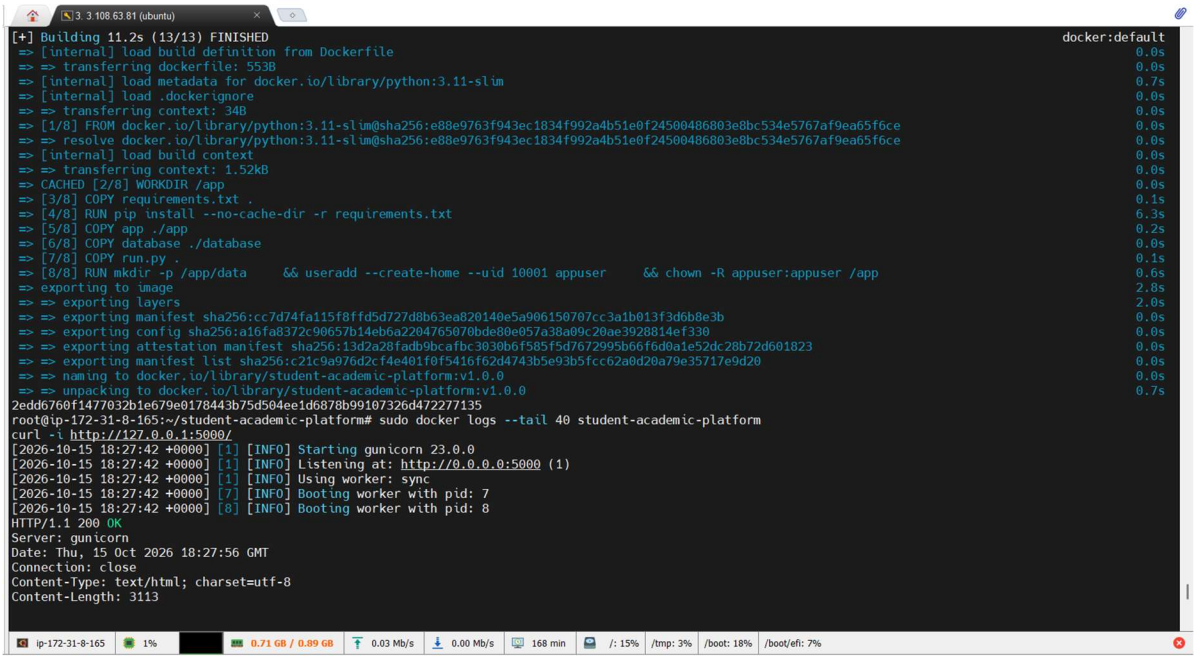
Screenshot 26



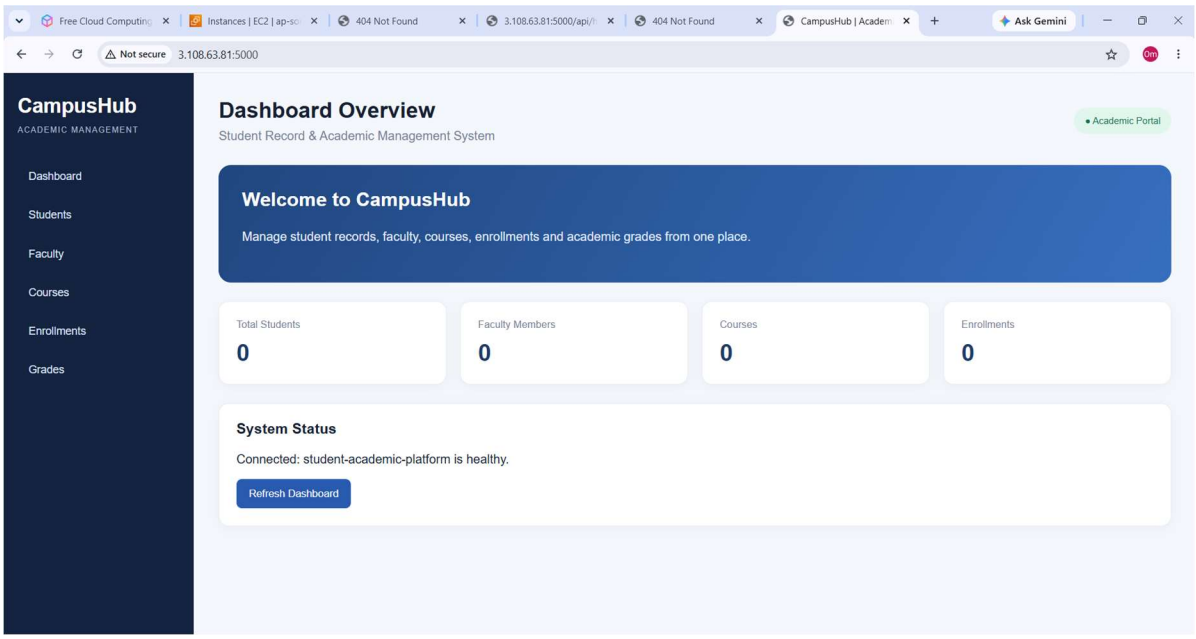
Screenshot 27

```
sudo docker images
[+] Building 13.3s (13/13) FINISHED                                                
```

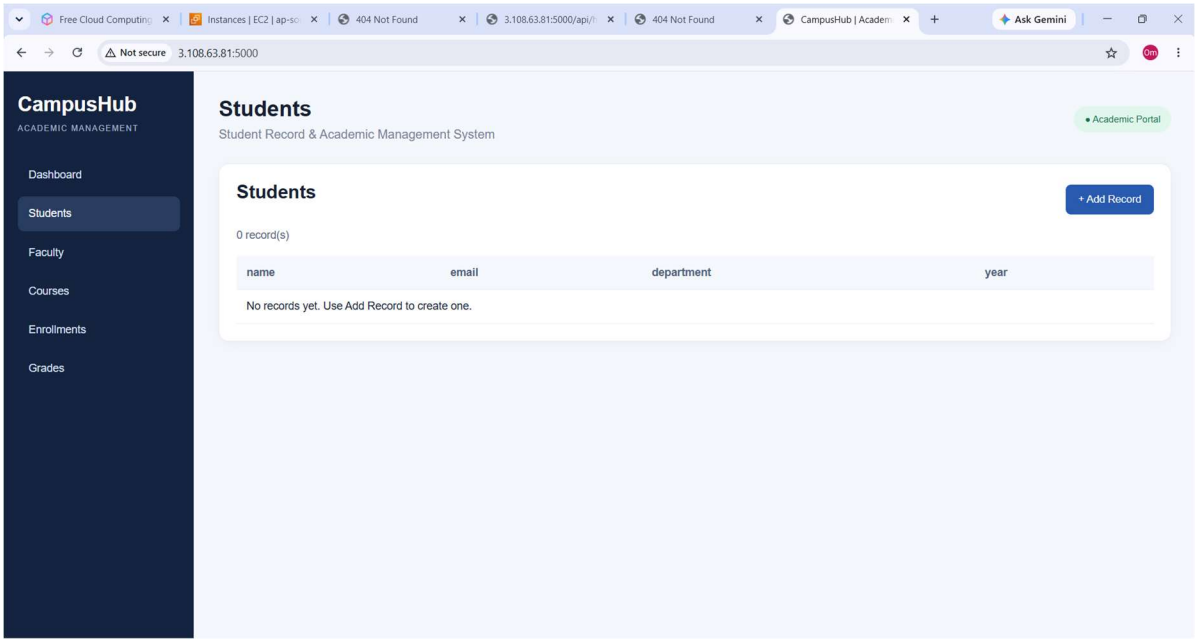
Screenshot 30



Screenshot 31



Screenshot 32


Free Cloud Computing
Instances | EC2 | ap-so
404 Not Found
3.108.63.81:5000/api/
404 Not Found
CampusHub | Academ
Ask Gemini
Not secure
3.108.63.81:5000
CampusHub
ACADEMIC MANAGEMENT
Dashboard
Students
Faculty
Courses
Enrollments
Grades
Students
Student Record & Academic Management System
• Academic Portal
Students
+ Add Record
0 record(s)
name
email
department
year
No records yet. Use Add Record to create one.